2026-2 Midterm

Computer Architecture · Midterm Study Notes

4 units · 22 lessons · same content as the trainer's Learn tab (English)

U1 Ch.1 Computer Abstractions and Performance

L1-1 · Computer Components · Abstraction · ISA

L1-2 · Defining Performance · Execution Time · Clock

L1-3 · Instruction Count · CPI · The Performance Equation

L1-4 · Power · Amdahl · The MIPS Pitfall · RISC vs CISC

U2 Ch.2 Instructions: Language of the Computer (MIPS)

L2-1 · Arithmetic Instructions · Register Operands

L2-2 · Memory Operands · Endianness · lw/sw

L2-3 · Constants · Two's Complement · Sign Extension · Hexadecimal

L2-4 · Instruction Encoding: R-format · I-format

L2-5 · Logical Operations · Conditional Branches · if/while

L2-6 · Procedure Calls · The Stack

L2-7 · Addressing · 32-bit Constants · Translating and Linking

U3 Ch.3 Arithmetic for Computers (Integer · Floating Point)

L3-1 · Addition · Subtraction · Overflow

L3-2 · Multiplication Hardware

L3-3 · Division Hardware

L3-4 · IEEE 754 Floating-Point Representation

L3-5 · Floating-Point Addition · Multiplication · FP Instructions

U4 Ch.4 The Processor: Datapath and Control (Simplified Version)

L4-1 · Processor Overview · Instruction Execution Steps

L4-2 · Logic Design Basics · Clocking

L4-3 · Datapath Elements

L4-4 · Tracing the Full Datapath (R-type · lw · sw · beq)

L4-5 · ALU Control

L4-6 · The Main Control Unit · Control Signal Table

U1 · week1-orientation.pdf · W02 Performance.pdf

Ch.1 Computer Abstractions and Performance

The exam is written in English and answered in English. This unit is half short-answer (defining terms) and half calculation (CPU time · CPI · Amdahl · MIPS). The professor said the calculation problems will be "almost the same as the class examples", so make sure you can solve every slide example by hand.

Computer Components · Abstraction · ISA

Slides: W02 p.2–16

§ Course Scope and Exam Format

- Textbook: Computer Organization and Design (MIPS). But questions come **only from the lecture materials**.
- **All lectures and exams are in English. You must answer in English too.**
- Grading: attendance (random roll call) 10%, **midterm 45%**, final 45%. Both midterm and final are closed book and offline.
- Midterm scope: Performance → Instructions → Arithmetic → Processor (simplified, single-cycle version). Pipelining is on the final.

Exam point: The midterm is Tue 10/20, 17:00–19:00. If you cannot attend, email the professor.

Lecture note `Week 1-1`
All lectures and exams are in English, and you must **answer in English** too. Both midterm and final are closed book and offline. 'Up to the midterm it is relatively easy; after the midterm, from pipelining on, it gets harder.' Attendance is not called every time; it is a random roll call.

§ Classes of Computers and the Eight Great Ideas

Class	Purpose	Characteristics
Supercomputer	High-end scientific and engineering computation	Measured in **FLOPS** (> MIPS), ~10^17
Server	Network-based, high-performance processing	Capacity, performance, reliability
Personal computer	General purpose	Cost/performance trade-off
Embedded	Hidden inside a system	Strictest power, performance and cost constraints

Eight Great Ideas: Design for Moore's Law · Use abstraction to simplify design · **Make the common case fast** · Hierarchy of memories · Dependability via redundancy · Performance via parallelism · Performance via pipelining · Performance via prediction.

Lecture note `Week 1-2`
Supercomputers are measured in FLOPS and are about 10,000 times faster than an ordinary desktop. Of the eight great ideas, memory hierarchy and prediction belong to the final; parallelism and pipelining connect to pipelining.

§ Below Your Program · Translation Steps

- **Application software**: written in a high-level language (HLL) (browsers, apps, Office).

- **System software**: **Compiler** (translates HLL into machine code, e.g., GCC) + **Operating System** (handles input/output, manages memory and storage, schedules tasks and shares resources).
- **Hardware**: processor, memory, I/O controllers.

Translation: `A+B` (HLL) → `add A, B` (assembly, still human-readable) → binary machine language.

What the OS does for an application: loading and process creation, **system call**, resource management (memory, CPU scheduling, I/O), IPC, event handling, error handling. You do not need to memorize the slide; just know "what the OS does in the flow of an application".

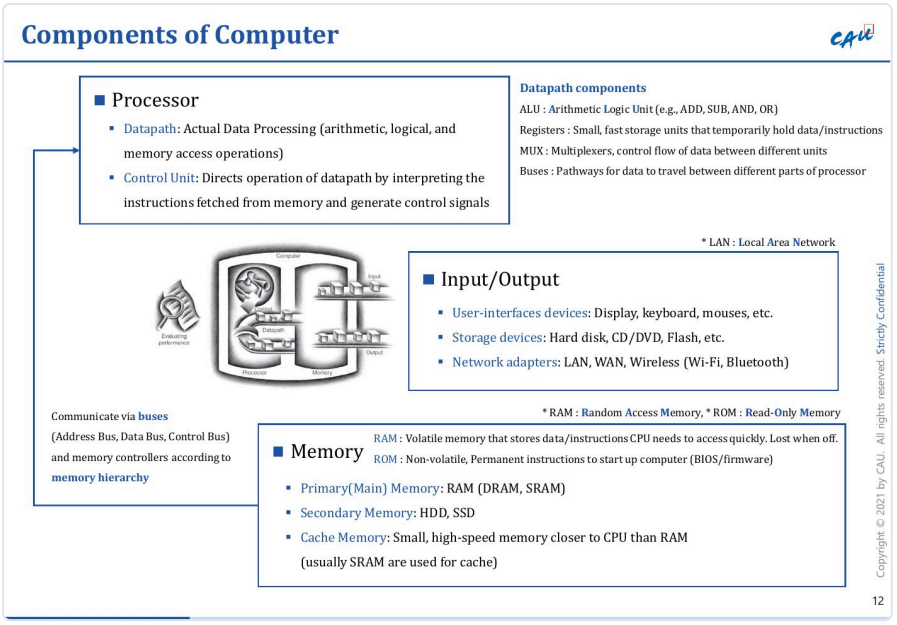
> **Lecture note** `Week 1-2`
> No need to memorize the OS slides; just know 'what the OS does in the flow of an application'. Example: when you send a photo in KakaoTalk and the 'Allow access?' prompt appears, that is the OS's job (a system call).

§ Components of a Computer

- **Input/Output**: user devices (keyboard, mouse), storage devices (HDD, Flash), and **network adapters are also I/O** (LAN, Wi-Fi, Bluetooth).
- **Memory**: Primary (RAM: DRAM/SRAM, volatile), Secondary (HDD, SSD), **Cache** (small and fast, usually SRAM). ROM = non-volatile, holds boot firmware (BIOS).
- **Processor = Datapath + Control unit**
  - **Datapath**: actual data processing (arithmetic, logical, and memory access operations). Components: ALU, Registers, MUX, Buses.
  - **Control unit**: directs the operation of the datapath by interpreting the instructions fetched from memory and generating **control signals**.
- Devices communicate over a **bus** (address bus, data bus, control bus).

> Exam point: "The two components of a processor and their roles" → Datapath (performs the actual data processing), Control unit (interprets instructions → generates control signals).

> **Lecture note** `Week 1-2`
> Many students get confused that network devices are also I/O. When the Control unit reads an instruction (e.g., an addition), it sends signals so that the ALU in the datapath operates. Cache is the memory to watch most carefully in this course (final).



W02 p.13 · Components of Computer: I/O, Memory, Processor (Datapath + Control)

§ Abstraction and ISA

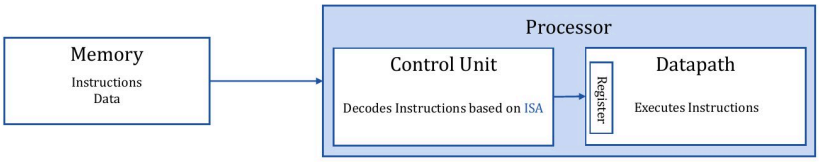
ISA (Instruction Set Architecture): the **interface** between hardware and low-level software. A **contract** that specifies how the processor interprets and executes instructions, which registers exist, and how data is accessed.

- Allows developers to write software without needing to understand the detailed inner workings of the hardware (abstraction = hide lower-level detail).
- **Portability**: software does not need to change when the hardware changes. The professor called this the most important property.
- Register = the fastest memory, located inside the processor.

One-line answer: ISA is the interface between hardware and low-level software; it defines how instructions are executed, which registers exist, and how data is accessed, so software can run without knowing hardware details (portability).

> **Lecture note** `Week 1-2`
> Coffee-errand analogy: the single request 'go buy me a coffee' hides 'open the door → elevator → 1st floor → order → pay → come back up'. The ISA hides hardware details in the same way. The most important property is **portability**.

Abstractions and ISA



Register : Fastest memory in computer system located directly within the processor.

- **Abstraction** helps us deal with complexity
  - Hide lower-level detail
- Instruction Set Architecture (ISA)
  - Interface between hardware and low-level software
  - Very important abstraction (How a processor interprets and executes instructions).
  - Allowing developers to write software without needing to understand detailed inner workings of the hardware.
  - Portability: Hardware can evolve without requiring changes in the software.
  - Contract of how instructions are executed, what registers are available, how data is accessed and manipulated.

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

W02 p.16 · Abstractions and ISA

Defining Performance · Execution Time · Clock

Slides: W02 p.17–26

§ Factors That Affect Performance

Component	What it affects
Algorithm	**Number of operations** executed (e.g., bubble sort n^2 vs merge sort n log n)
Programming language, compiler, architecture	Number of **machine instructions per operation**
Processor & memory system	How fast instructions are executed
I/O system (including OS)	How fast I/O operations are executed

Moore's Law: IC capacity doubles every 18–24 months. But because of power constraints (the power wall), performance cannot rise forever.

> **Lecture note** `Week 1-2 · Week 2-1`
> If you replace an n log n algorithm with an n^2 one, it becomes slower no matter how fast the processor is. Total machine instructions = number of operations × machine instructions per operation. The same code usually runs faster in C than in Python.

§ Response Time vs Throughput

- **Response time** (= latency = execution time): how long it takes to do a task/job.
- **Throughput**: total work done per unit time (e.g., tasks/hour).
- In this course we **focus on response time**.

> **Lecture note** `Week 1-2`
> Response time is measuring with a stopwatch until one task finishes; throughput is how many tasks are processed per unit time. The course uses response time throughout.

§ Comparing Performance: X Is n Times Faster Than Y

`Performance_X = 1 / Execution time_X`

`Performance_X / Performance_Y = Execution time_Y / Execution time_X = n` → "X is n times faster than Y".

Example (slide): the same program takes 7 s on A and 21 s on B → Perf_A/Perf_B = 21/7 = **3**, so A is 3 times faster than B.

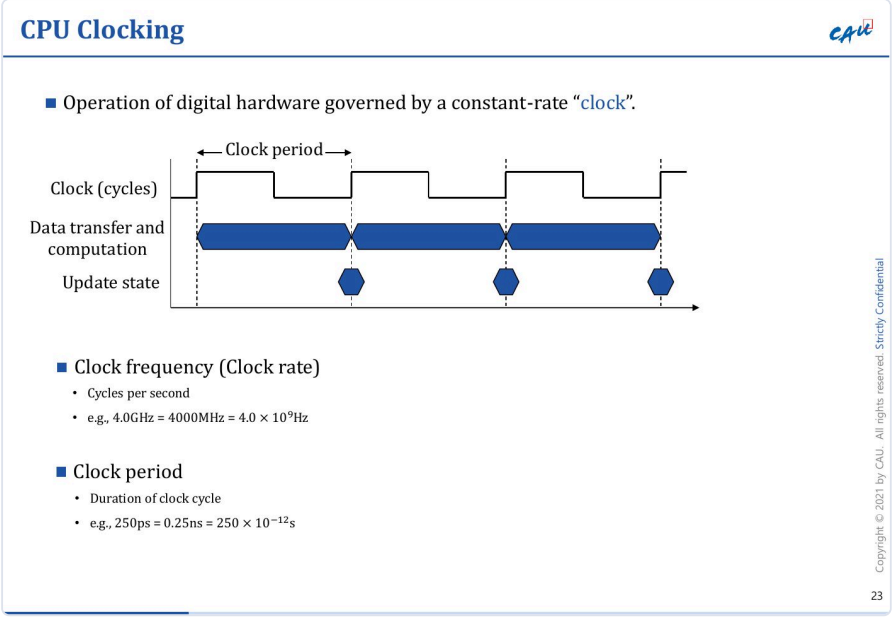
Trap: Do not flip the ratio. The performance ratio is the **inverse** of the time ratio. The professor stressed twice that "many students miss this in their midterm exams".

Lecture note `Week 1-2 · Week 2-1`
'many students miss this in their midterm exams' — the performance ratio is the **inverse** of the execution-time ratio. He had students solve the 7 s vs 21 s example themselves for 2 minutes and confirmed the answer 3.

§ Elapsed Time · CPU Time · Clock

- **Elapsed time**: total response time, including all aspects (I/O waits, OS overhead). Determines system performance, but often not good for comparison purposes.
- **CPU time** = user CPU time + system CPU time.
  - **User CPU time**: time spent actually running the instructions of our program (user mode). **In this course, execution time = user CPU time.**
  - System CPU time: kernel mode (hardware access, I/O, memory management).
- **Clock**: an internal signal that toggles 0→1→0 at a fixed period.
  - **Clock period** (cycle time): duration of one clock cycle, e.g., 250 ps.
  - **Clock rate** (frequency): cycles per second, e.g., 4.0 GHz = 4.0×10^9 Hz. **The two are reciprocals** (4 GHz ↔ 250 ps).

Lecture note `Week 2-1`
Elapsed time includes I/O delays and the delay of moving data from memory to the processor, so it is inaccurate for comparison. That is why we look at CPU time, and only the **user CPU time** within it. From now on, 'execution time' means user CPU time.



W02 p.24 · CPU Clocking: clock period and clock rate

§ The CPU Time Formula and Example

`CPU Time = CPU Clock Cycles × Clock Cycle Time = CPU Clock Cycles / Clock Rate`

To improve performance: reduce the number of clock cycles or increase the clock rate. There is often a trade-off between the two.

Slide example: A runs in 10 s at 2 GHz. B should run in 6 s, but needs 1.2 times as many clock cycles as A.

```
Cycles_A = 10 s × 2×10⁹ Hz = 20×10⁹
Cycles_B = 1.2 × 20×10⁹ = 24×10⁹
Clock rate_B = 24×10⁹ / 6 s = 4×10⁹ Hz = 4 GHz
```

Exam point: In the professor's own words, "the examples that I provided during class will resemble very much your midterm". Practice rearranging the formula to solve for the unknown.

Lecture note `Week 2-1`

'the examples that I provided during class will resemble very much your midterm or your final exam' — he said if you can solve the examples, you can solve the exam problems. He solved it by rearranging the formula to clock rate = cycles / CPU time.

Instruction Count · CPI · The Performance Equation

Slides: W02 p.27–31

§ IC · CPI · The Three-Factor Formula

- **Instruction Count (IC)**: number of instructions a program executes. Determined by the program, the ISA and the compiler.
- **CPI (Cycles Per Instruction)**: average number of clock cycles per instruction. Determined by the CPU hardware. If instructions have different CPIs, use the average CPI.

`Clock Cycles = IC × CPI`

`CPU Time = IC × CPI × Clock Cycle Time = (IC × CPI) / Clock Rate`

Slide example: A has cycle time 250 ps and CPI 2.0, B has 500 ps and CPI 1.2, same program (IC = I).

```
CPU time_A = I × 2.0 × 250 ps = 500·I ps
CPU time_B = I × 1.2 × 500 ps = 600·I ps
Perf_A / Perf_B = 600I / 500I = 1.2  → A is 1.2 times faster than B
```

Exam point: If the IC is unknown, call it I; it cancels out in the ratio. This is exactly the method the professor used.

Lecture note Week 2-1
If the IC is unknown, use the same variable i for both, and it cancels when you divide the two computers. CPI is the most direct measure of CPU performance.

§ Weighted Average CPI

If different instruction classes have different CPIs:

`Clock Cycles = Σ (CPI_i × IC_i)`

`Average CPI = Clock Cycles / IC = Σ CPI_i × (IC_i / IC)` (IC_i / IC = relative frequency)

Slide example (CPI: A=1, B=2, C=3):

	A	B	C	Total IC	Cycles	Avg. CPI
Sequence 1	2	1	2	5	2+2+6 = 10	**2.0**
Sequence 2	4	1	1	6	4+2+3 = 9	**1.5**

Sequence 2 executes more instructions but fewer cycles, so it is faster.

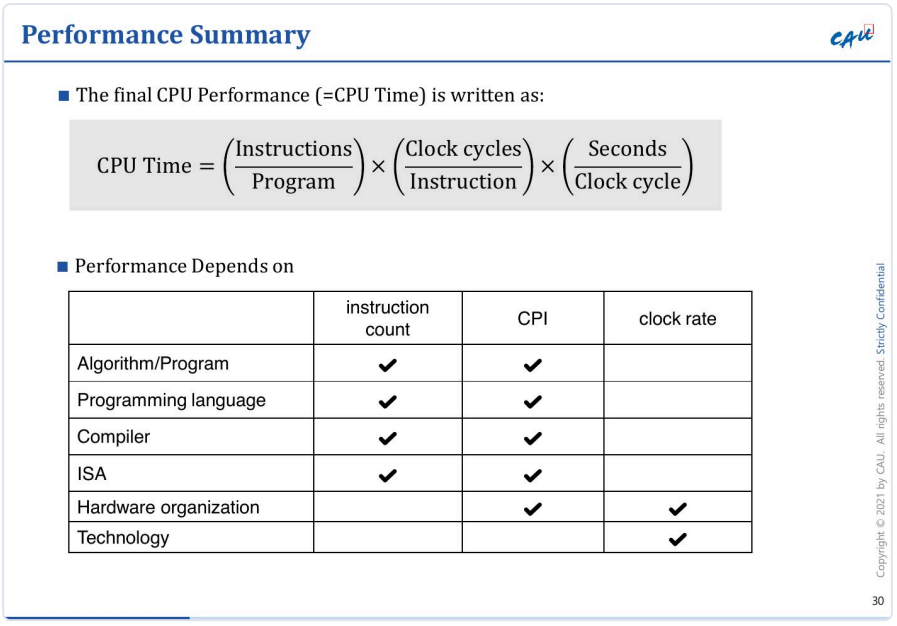
Lecture note Week 2-1
Order of solution: first count the total number of instructions → add up CPI × IC for each class → divide by the total number of instructions.

§ Performance Summary Table

`CPU Time = (Instructions / Program) × (Clock cycles / Instruction) × (Seconds / Clock cycle)`

	IC	CPI	Clock rate
Algorithm/Program	✔	✔	
Programming language	✔	✔	
Compiler	✔	✔	
ISA	✔	✔	
Hardware organization		✔	✔
Technology			✔

Trap: Hardware organization affects CPI and clock rate, but not IC. Technology affects only the clock rate.



W02 p.31 · Performance Summary table

Power · Amdahl · The MIPS Pitfall · RISC vs CISC

Slides: W02 p.32–41

§ The Power Wall

CMOS: `Power ∝ 1/2 × Capacitive load × Voltage² × Frequency`

Example: a new CPU has 85% of the capacitive load, 15% lower voltage and 15% lower frequency → P_new/P_old = 0.85 × 0.85² × 0.85 = 0.85⁴ ≈ **0.52**.

Power wall: we cannot reduce the voltage further and cannot remove more heat → single-processor performance improvement stalls → **multicore** (requires explicit parallel programming and load balancing). The professor said you do not need to memorize the formula; just know that "power is constrained".

> **Lecture note** `Week 2-1 · Week 2-2`
> No need to memorize the formula; just take away that 'power is not infinite'. Power wall: we cannot lower the voltage any further and cannot remove any more heat.

§ Amdahl's Law

Pitfall: Expecting the improvement of one aspect of a computer to increase overall performance by a proportional amount.

`T_improved = T_affected / improvement factor + T_unaffected`

Slide example: of 100 s, multiplication takes 80 s. How much must multiplication improve to make the whole program 5 times faster (20 s)?

```
20 = 80/n + 20  →  80/n = 0  → impossible
```

Even with infinitely fast multiplication, the remaining 20 s is the bottleneck, so the speedup is capped at 5×. → Connects to **Make the common case fast**.

> **Lecture note** `Week 2-2`
> Even if multiplication is made infinitely fast, the remaining 20 seconds are the bottleneck, so the maximum is 5×. 'this is why our computers are still very slow'.

§ MIPS Is a Poor Performance Metric

`MIPS = IC / (Execution time × 10⁶) = Clock rate / (CPI × 10⁶)`

MIPS does not account for (1) differences in ISA between computers or (2) differences in instruction complexity. The IC cancels out and disappears.

Slide example (both 4 GHz):

	IC	CPI	MIPS	CPU time
A	10 billion	1.0	4000	$10\times10^9\times1.0 / 4\times10^9 = 2.5$ s
B	8 billion	1.1	≈3636	$8\times10^9\times1.1 / 4\times10^9 = 2.2$ s

A has the higher MIPS, but **B is actually faster** (Perf_A/Perf_B = 2.2/2.5 = 0.88). That is why we use CPU time.

> **Lecture note** `Week 2-2`
> MIPS cancels out the instruction count, so the comparison goes wrong. In the example A has the higher MIPS, but by CPU time 8.8/10 < 1 → B is faster.

§ Other: SPEC · Power at Idle

- **SPEC** (Standard Performance Evaluation Corp.): standard benchmark suites for CPU, I/O, Web, etc. SPEC CPU measures the elapsed time of programs with negligible I/O; SPEC Power measures power at different load levels.
- **Fallacy: low power at idle** — an i7 draws 121 W (47% of peak) even at 10% load. Data centers mostly run at 10–50% load → processors should be designed so that power is proportional to load.

§ RISC vs CISC

	RISC	CISC
Examples	**MIPS**, ARM, PowerPC	**x86, x86-64** (Intel, AMD)
Strategy	Simple, small instruction set	Large, complex instructions that minimize instructions per program
Advantages	Fast execution, power efficiency, scalability (easy to parallelize)	Smaller programs, compatibility with old instructions
Disadvantages	Complex tasks need more instructions → larger programs	Slower execution, higher power consumption

MIPS = Microprocessor without Interlocked Pipeline Stages. (Same name as the MIPS performance metric, nothing else in common!)

> **Lecture note** `Week 2-2 · Week 3-1`
> The x86/x64 shown in Windows System Information indicates CISC. RISC is the side where you have to give every command one by one, like 'open the door, take the elevator...'; CISC bundles them into one big instruction.

U2 · W02 p.41 · week3-수업.pdf · week4-수업.pdf p.2–5

Ch.2 Instructions: Language of the Computer (MIPS)

The biggest chunk of the exam. The announced format is "given an instruction and a description, fill in the blank or write the MIPS expression". This unit has many rules the professor warned would get zero points, so be sure to read the Trap boxes.

Arithmetic Instructions · Register Operands

Slides: week3 p.3–8

§ Classes of MIPS Instructions

MIPS: a RISC ISA from Stanford, with a large share of the embedded market; typical of many modern ISAs.

- **By type**: Arithmetic · Memory (Data Transfer) · Logical · Conditional · Branch/Jump · Floating Point · Pseudo
- **By format**: **R-format · I-format · J-format** · FP format

§ Arithmetic Instruction Format

`add a, b, c # a = b + c`

- **One destination + two sources**. All arithmetic instructions have this form.
- **Design Principle 1: Simplicity favors regularity** — regularity makes implementation simpler; simplicity enables higher performance at lower cost.

Slide example:

```
a = b + c + d;          f = (g + h) - (i + j);
add a, b, c             add t0, g, h
add a, a, d             add t1, i, j
                        sub f, t0, t1
```

Trap: The result must go into **the variable the user asked for** (here, f). The professor stressed "must be very careful at this term".

> **Lecture note** Week 2-2
> a = b + c + d cannot be written as a single add; split it into two lines. The result must go into the variable the user wanted — 'you must take a careful look at where to put your result'.

§ Register Operands

- Arithmetic instructions use **register operands only**. A register is a special storage location inside the processor.
- MIPS has **32 × 32-bit registers** (numbered 0–31). 32-bit = word size (the standard when MIPS was designed, kept in MIPS64 for compatibility).
- Assembler names: `$t0-$t9` (temporaries), `$s0-$s7` (saved).
- **Design Principle 2: Smaller is faster** — we design very small registers for faster operations on frequently accessed data.

Slide example (f, g, h, i, j in $s0–$s4):

```
add $t0, $s1, $s2     # g + h
add $t1, $s3, $s4     # i + j
sub $s0, $t0, $t1     # f = (g+h) - (i+j)
```

Trap: An operation's result cannot go directly to memory. It is always stored **in a register**. (This slide had an add/sub typo, which the professor corrected in class: "change this into subtraction".)

Lecture note `Week 2-2`

No operation works directly on data in memory: load into a register → operate → store. Results can only be written to registers. The last add on the slide is a typo for sub.

Memory Operands · Endianness · lw/sw

Slides: week3 p.10–14

§ Memory Basics

- Main memory is used for **composite data** such as arrays, structures and dynamic data. There are too few registers to hold every variable.
- Before an arithmetic operation, **load** (memory → register); afterwards, **store** (register → memory).
- **Byte addressed**: each address identifies one 8-bit byte.
- **Words are aligned**: the address of a word (32-bit) must be a **multiple of 4**.
- Address of an array element = base address + index × 4 (int array).

> **Lecture note** `Week 2-2`
> Memory is aligned, so think of it like an array: desired address = base + index × 4.

§ Big Endian vs Little Endian

- **Big endian**: the MSB (most significant byte) is stored at the **lowest memory address**.
- **Little endian**: the LSB (least significant byte) is stored at the **lowest memory address**.

0x12345678 stored at addresses 0–3:

Address	0	1	2	3
Big endian	12	34	56	78
Little endian	78	56	34	12

Trap: Two hex digits are **1 byte**. Even in little endian, do not reverse the two digits inside a byte (writing 78 as 87 is wrong). The professor stressed this twice.

> **Lecture note** `Week 2-2`
> Two hex digits make one number (byte). Even in little endian, **never swap the order of the two digits** (writing 78 as 87 is wrong).

§ lw / sw

`lw dest, offset(base)` · `sw src, offset(base)`

- Three operands: a register, an **offset (constant, in bytes)**, and a base address (register).
- In sw the first register is the **source** (the value written to memory).

Slide example 1: `g = h + A[8];` (g:$s1, h:$s2, base of A:$s3)

```
lw  $t0, 32($s3)      # offset = 8 × 4 = 32
add $s1, $s2, $t0
```

Slide example 2: `A[12] = h + A[8];`

```
lw  $t0, 32($s3)
add $t0, $s2, $t0
sw  $t0, 48($s3)      # 12 × 4 = 48
```

Trap: **The offset must be a constant.** Putting a register in the offset position gets **zero points** even if the logic is right, and in a True/False question it is always False. If the index is a variable, compute the address first (`sll`, `add`), then use `lw $t0, 0($t1)`.

Lecture note `Week 2-2 · Week 3-1`
In lw the first variable is the destination; in sw the first variable is the **source**. offset = index × word size (4).

§ Registers vs Memory

- Registers are much faster than memory and easier to use.
- Operating on memory data requires loads and stores → more instructions executed.
- The compiler must use registers for variables as much as possible and only **spill** to memory for less frequently used variables. Register optimization is important for efficiency.

Constants · Two's Complement · Sign Extension · Hexadecimal

Slides: week3 p.15–23

§ Immediate Operands and $zero

`addi $s3, $s3, 4` — the constant is placed directly in the instruction.

- **There is no subtract immediate** → use addi with a negative constant: `addi $s2, $s1, -1`.
- **Design Principle 3: Make the common case fast** — small constants are common, and an immediate operand avoids a load instruction.
- **$zero (register 0)** = always the constant 0, **cannot be overwritten**.
- Copying a register (move): `add $t2, $s1, $zero`

Trap: A copy is **`add` + `$zero`**. Using `addi` with `$zero` (a register) as if it were the constant gets zero points (the professor, Week 4-2). `$zero` is a register, not a constant.

Lecture note Week 3-1 · Week 4-2
There is no subtract immediate; use addi with a negative constant. $zero cannot be overwritten. Copying a register is add + $zero, and 'you should never use the addition immediate when adding zero ... you will get zero points'.

§ Unsigned and Two's Complement

- **Unsigned** n-bit: $x = x_{n-1}2^{n-1} + \ldots + x_0 2^0$, range **0 to $2^n - 1$** (32-bit max 4,294,967,295).
- **2's complement** n-bit: $x = -x_{n-1}2^{n-1} + x_{n-2}2^{n-2} + \ldots + x_0 2^0$, range **$-2^{n-1}$ to $2^{n-1} - 1$**.
  - bit 31 = sign bit (1 = negative, 0 = non-negative). Non-negative numbers have the same representation as unsigned.
  - `1111…1111` = −1, `1000…0000` = −2,147,483,648, `0111…1111` = 2,147,483,647.

Lecture note Week 3-1
The first bit is the sign: 0 means positive, 1 means negative. Adding x and $\bar{x}$ gives all 1s = −1, so $-x = \bar{x} + 1$.

§ Signed Negation: Complement and Add 1

Since $x + \bar{x} = 1111\ldots1111_2 = -1$, we get $\bar{x} + 1 = -x$.

Example: +2 = `0000 … 0010` → invert `1111 … 1101` → +1 → `1111 … 1110` = −2.

Class example: −10 (32-bit)

```
+10 = 0000 0000 0000 0000 0000 0000 0000 1010
inv = 1111 1111 1111 1111 1111 1111 1111 0101
 +1  = 1111 1111 1111 1111 1111 1111 1111 0110   (= 0xFFFFFFF6)
```

Lecture note Week 3-1
−10 example: write +10 in 32 bits → flip every 0 and 1 → add 1. The result starts with 1, confirming it is negative.

§ Sign Extension

Representing a number using more bits while **preserving its value**: **replicate the sign bit to the left**.

- Used in MIPS by: `addi` (extends the immediate), `lb` · `lh` (extend the loaded byte/halfword), `beq` · `bne` **(extend the displacement)**.
- 8-bit → 16-bit: +2 = `0000 0010` → `0000 0000 0000 0010`, −2 = `1111 1110` → `1111 1111 1111 1110`.

Trap: Slide p.21 shows the extended −2 as `1111 1111 1000 0010`; that is a **typo**. The correct value is `1111 1111 1111 1110`.

> **Lecture note** `Week 3-1`
> When extending a smaller bit width to a larger one, fill the upper bits with the sign. He foreshadowed that 'this becomes more important later in the semester' → actually used in the Week 6 datapath (lw offset, beq).

§ Hexadecimal

One hex digit = 4 bits. Table: 0=0000 … 9=1001, a=1010, b=1011, c=1100, d=1101, e=1110, f=1111.

Example: `eca8 6420` → `1110 1100 1010 1000 0110 0100 0010 0000`

Instruction Encoding: R-format · I-format

Slides: week3 p.22–28

§ Representing Instructions

- Instructions are encoded as **32-bit binary numbers (machine code)**. Keep the number of formats small → **regularity**.
- **opcode**: the first 6 bits. It tells which format the instruction has, so the rest can be interpreted.
- Register numbers: `$t0-$t7` = 8–15, `$s0-$s7` = 16–23, `$t8-$t9` = 24–25 (others: $zero 0, $v0–$v1 2–3, $a0–$a3 4–7, $gp 28, $sp 29, $fp 30, $ra 31).

§ R-format

op	rs	rt	rd	shamt	funct
6 bits	5	5	5	5	6 bits

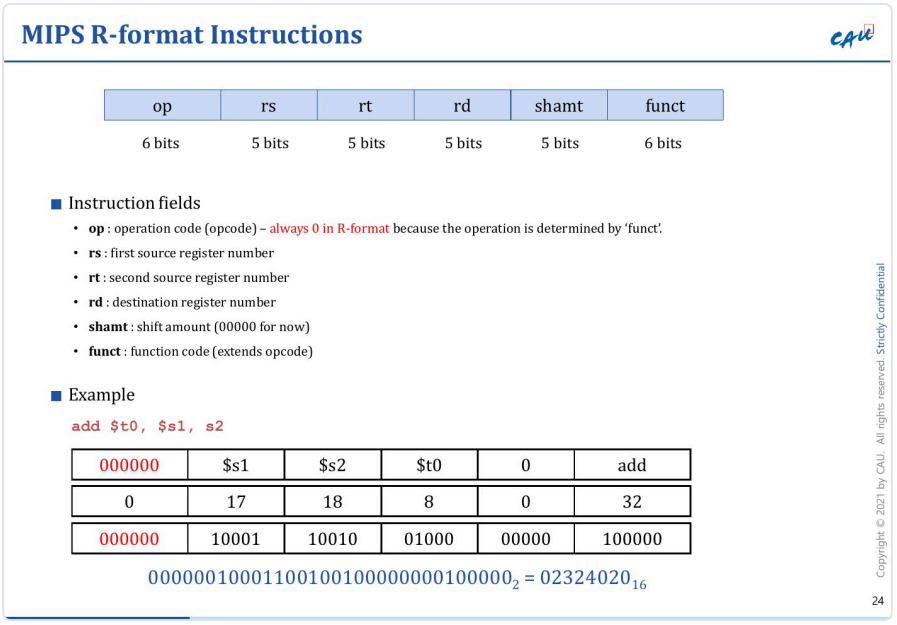
- **op = always 0** (the funct field selects the operation).
- rs: first source register, rt: second source register, **rd: destination register**, shamt: shift amount (0 for now), funct: function code (extends the opcode).
- funct values are **given** in the exam (no need to memorize). For reference: add 32, sub 34, and 36, or 37, slt 42.

Example: `add $t0, $s1, $s2`

```
op      rs     rt     rd     shamt  funct
000000  10001  10010  01000  00000  100000
0       17     18     8      0      32
= 0000 0010 0011 0010 0100 0000 0010 0000₂ = 0x02324020
```

Trap: Assembly order is **dest, src, src**, but machine code order is **rs, rt, rd** (sources first, destination last). The professor stressed this in both Week 3 and Week 6.

> **Lecture note** `Week 3-1`
> 'You should mind that the order is twisted' — assembly puts dest first; machine code is rs, rt, rd. funct values are given on the exam, so no need to memorize them.



week3 p.25 · R-format fields and the encoding of add $t0,$s1,$s2

§ I-format

op	rs	rt	constant or address
6 bits	5	5	16 bits

- Immediate arithmetic and load/store. **rt = destination (lw, addi) or source (sw)**.
- constant: -2^{15} to $+2^{15} - 1$. address: an offset added to the base address in rs.
- Examples: addi, lw, sw, beq, bne, slti, andi, ori, lui. Opcodes for reference: lw 35, sw 43, beq 4, bne 5, addi 8.
- **Design Principle 4: Good design demands good compromises** — different formats complicate decoding, but allow all instructions to be 32 bits uniformly. Keep formats as similar as possible.

> **Lecture note** `Week 3-1`
> I-format has no rd or shamt, so 16 bits can be used for the constant. That is why sign extension is needed to match 32 bits.

§ Encoding Example: A[300] = h + A[300]

`$t1` = base of A, `$s2` = h.

```
lw  $t0, 1200($t1)     # 300 × 4 = 1200
add $t0, $s2, $t0
sw  $t0, 1200($t1)
```

Instruction	op	rs	rt	rd	shamt	funct / address
lw	35	9	8			1200
add	0	18	8	8	0	32
sw	43	9	8			1200

Binary: lw = `100011 01001 01000 0000 0100 1011 0000`; sw differs only in op, `101011`.

Lecture note `Week 3-1`

A[300] example: 300 × 4 = 1200. lw opcode 35, sw opcode 43 (decimal). This is how assembly is encoded into machine code.

Logical Operations · Conditional Branches · if/while

Slides: week3 p.30–40 · week4 p.2

§ Shift Operations

Operation	C	Java	MIPS		
Shift left	<<	<<	**sll**		
Shift right	>>	>>>	**srl**		
Bitwise AND/OR/NOT	& |	~	& |	~	and, andi / or, ori / **nor**

- The number of positions to shift goes in the R-format **shamt** field.
- **sll by i** = multiply by 2^i (fills the right with 0s). **srl by i** = divide by 2^i, **unsigned only**.

Trap: srl fills the left with 0s, so using it on a negative (two's complement) number destroys the sign (the professor: "you should not just perform this operation on a signed number").

> **Lecture note** `Week 3-1`
> A right shift fills with 0s, so using it on signed (two's complement) numbers breaks the sign.

§ AND · OR · NOR

- **AND**: masking. Keeps the selected bits and clears the rest to 0.
- **OR**: sets the selected bits to 1 and leaves the rest unchanged.
- MIPS has no **NOT** → use **3-operand NOR**: `nor $t0, $t1, $zero` = NOT $t1.

Example ($t1 = 0...0011 1100 0000 0000, $t2 = 0...0000 1101 1100 0000):

- `and $t0,$t1,$t2` → 0...0000 1100 0000 0000
- `or $t0,$t1,$t2` → 0...0011 1101 1100 0000

§ Branch Instructions

- `beq rs, rt, L1`: if rs == rt, branch to L1.
- `bne rs, rt, L1`: if rs != rt, branch to L1.
- `j L1`: unconditional jump to L1.

§ Compiling if-else

`if (i == j) f = g + h; else f = g - h;` (f, g, h, i, j = $s0–$s4)

```
      bne $s3, $s4, Else    # if i ≠ j, go to Else
      add $s0, $s1, $s2     # f = g + h
      j   Exit
Else: sub $s0, $s1, $s2     # f = g - h
Exit:
```

The trick is to test the **opposite** condition and skip to Else (if i == j, bne is not taken, so the if-block runs).

Lecture note Week 4-1

If i == j, bne is not taken, so the if-block runs and then unconditionally j Exit. If bne is taken (i ≠ j), the else-block's sub runs.

§ Compiling a while Loop

`while (save[i] == k) i += 1;` (i: $s3, k: $s5, base of save: $s6)

```
Loop: sll  $t1, $s3, 2      # t1 = i * 4
      add  $t1, $t1, $s6    # t1 = &save[i]
      lw   $t0, 0($t1)      # t0 = save[i]
      bne  $t0, $s5, Exit   # exit if save[i] ≠ k
      addi $s3, $s3, 1      # i += 1
      j    Loop
Exit:
```

Trap: Because the index i is a variable, you might want to write `lw $t0, $t1($s6)`, but that is **not allowed**. Compute the address first and use offset 0. In True/False, "a register as the offset" = False.

Lecture note Week 4-1

'if you face a true or false question where the offset is provided as a register, then you must put false' — that is why the address is computed first with sll and add, and the lw offset is 0.

§ slt / slti / sltu

- `slt rd, rs, rt`: if rs < rt then rd = 1, else 0.
- `slti rt, rs, constant`: if rs < constant then rt = 1.
- Combined with beq/bne, they build every comparison (==, !=, <, <=, >, >=).

```
slt $t0, $s1, $s2       # $t0 = 1 if $s1 < $s2
bne $t0, $zero, L       # branch to L if 1 ≠ 0
```

- **Signed vs Unsigned**: slt/slti = signed, sltu/sltiu = unsigned.
  With $s0 = 1111...1111 and $s1 = 0000...0001: `slt` → −1 < +1 → $t0 = 1; `sltu` → 4,294,967,295 > 1 → $t0 = 0.

Lecture note Week 4-1

slt/slti give 1 if the latter value is larger, otherwise 0.

§ Why There Is No blt or bge

- In hardware, < and > comparisons are slower than = and ≠.
- Combining them with a branch means more work per instruction → **slower clock → every instruction is penalized**.
- A good compromise: **beq, bne** + slt. (`blt` is an assembler pseudo-instruction that becomes `slt` + `bne`.)

Procedure Calls · The Stack

Slides: week3 p.42–51 · week4 p.2–5

§ Procedure Call Steps · jal / jr

1. Place parameters in registers → 2. Transfer control to the procedure → 3. Acquire storage → 4. Perform the operations → 5. Place the result in a register → 6. Return to the place of the call.

- `jal ProcedureLabel` (jump-and-link): **stores the address of the next instruction (PC+4) in $ra** and jumps to the target.
- `jr $ra` (jump register): copies $ra into the PC → return. Also used for computed jumps.
- PC: the register holding the address of the instruction currently being executed.

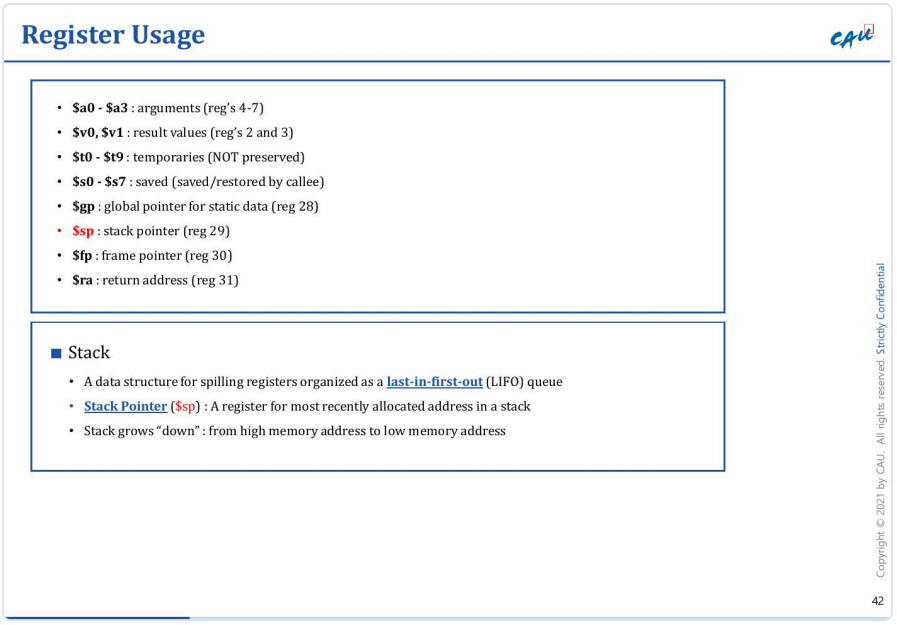
Example: `jal` at address 100, function at address 388 → $ra = 104, PC = 388. The `jr $ra` at the end of the function returns to 104.

> **Lecture note** `Week 4-1`
> jal on line 100 → $ra = 104 (the next line), PC = the line where the function is defined (e.g., 388). The function's jr $ra returns to 104.

§ Register Usage Conventions and the Stack

Register	Number	Use
$a0–$a3	4–7	Arguments
$v0, $v1	2–3	Result values
$t0–$t9	8–15, 24–25	Temporaries (**not preserved**)
$s0–$s7	16–23	Saved (**callee saves and restores**)
$gp / $sp / $fp / $ra	28 / 29 / 30 / 31	global ptr / **stack ptr** / frame ptr / **return address**

- **Stack**: a data structure for spilling registers, organized as a last-in-first-out (LIFO) queue. **$sp** holds the most recently allocated address in the stack.
- **The stack grows "down"** (from high memory address to low memory address): allocate space with `addi $sp, $sp, -N`.
- Procedure frame (activation record): local data allocated by the callee.



week3 p.43 · Register Usage and the stack

§ Leaf Procedure Example

A procedure that does not call other procedures. `int leaf_example(int g, h, i, j) { int f; f = (g+h) - (i+j); return f; }` (arguments in $a0–$a3, f in $s0)

```
leaf_example:
  addi $sp, $sp, -4      # space for one int (4 bytes)
  sw   $s0, 0($sp)       # save original $s0
  add  $t0, $a0, $a1     # g + h
  add  $t1, $a2, $a3     # i + j
  sub  $s0, $t0, $t1     # f
  add  $v0, $s0, $zero   # copy result to $v0
  lw   $s0, 0($sp)       # restore $s0
  addi $sp, $sp, 4
  jr   $ra
```

Lecture note `Week 4-1 · Week 4-2`

$s0 might be a register used elsewhere, so save it on the stack and restore it when leaving. One int, so $sp moves by 4. Moving the result into $v0 uses add + $zero.

§ Non-leaf (Recursive) Example: fact

A procedure that calls another procedure must save **$ra and any arguments needed afterwards** on the stack before the call, and restore them after the call.

```
fact:
  addi $sp, $sp, -8      # 2 slots: argument, return address
  sw   $ra, 4($sp)
  sw   $a0, 0($sp)
  slti $t0, $a0, 1       # n < 1 ?
  beq  $t0, $zero, L1    # if not, go to L1
  addi $v0, $zero, 1     # return 1
  addi $sp, $sp, 8
  jr   $ra
L1:
  addi $a0, $a0, -1      # n - 1
  jal  fact
  lw   $a0, 0($sp)       # restore from the same slot it was saved in
  lw   $ra, 4($sp)
  addi $sp, $sp, 8
  mul  $v0, $a0, $v0     # n * fact(n-1)
  jr   $ra
```

Trap: **The offset you save to and the offset you restore from must match.** If you saved the argument at 0 and $ra at 4, restore them the same way. Restoring them in the opposite order gets zero points (the professor, Week 4-1).

Trap: The C code on the Week 3 slide has a typo, `return f`. It is corrected to `return 1` on the Week 5 slide.

Lecture note `Week 4-1`
If you saved the argument at offset 0 and $ra at offset 4, restore from the same slots. 'if you ... load this on the opposite order, then you will get a zero point in your midterm'.

§ Tracing the Stack for fact(3)

Caller: `jal fact` at address 100 → $ra = 104, $a0 = 3. Initial $sp = 500.

Step	$sp	Saved at 0($sp)	Saved at 4($sp)
Enter fact(3)	492	a0 = 3	ra = 104
Enter fact(2)	484	a0 = 2	ra = L1+8
Enter fact(1)	476	a0 = 1	ra = L1+8
Enter fact(0)	468	a0 = 0	ra = L1+8

fact(0): slti → 0 < 1, so $t0 = 1 → beq not taken → $v0 = 1, $sp = 476, jr → L1+8.
On the way back, each `lw` restores a0 to 1, 2, 3 and $v0 goes 1 → 1 → 2 → 6. The last jr returns to **104**.

Trap: In `slti $t0, $a0, 1`, when a0 = 1, 1 < 1 is false → $t0 = 0 (recursion continues). This is the spot where the professor said "be extremely careful".

Lecture note `Week 4-1`
Traced fact(3) line by line starting from $sp 500: 492 → 484 → 476 → 468. At slti, 1 < 1 is false, so it recurses once more — 'be extremely careful'.

§ Stack Size by Argument Type (as taught by the professor)

- One int argument + $ra → 4 + 4 = **8 bytes**.
- **As the professor explained**: each char argument takes 1 byte. E.g., two char arguments + $ra → 4 + 1 + 1 = **6 bytes**; three chars → 4 + 3 = 7.
- If the value being saved is an int, you still move by 4 even when handling a char array (in strcpy the saved value is int i).

Caution: Real MIPS compilers usually round up to a multiple of 4 for alignment, but this is how it was explained in class. On the exam, follow the class rule.

Lecture note `Week 4-2 · Week 5-1`
Always check the argument types: char arguments take 1 byte each, so 2 chars + $ra = 6 bytes, 3 chars = 7 bytes. 'when actually facing your midterm, be especially careful about this format'.

§ Byte Operations · strcpy

- `lb rt, offset(rs)` / `lh`: sign-extend to 32 bits. `lbu` / `lhu`: zero-extend.
- `sb` / `sh`: store just the rightmost byte/halfword.

`void strcpy(char x[], char y[])` (addresses of x, y in $a0, $a1; i in $s0)

```
strcpy:
  addi $sp, $sp, -4
  sw   $s0, 0($sp)
  add  $s0, $zero, $zero   # i = 0
L1: add  $t1, $s0, $a1     # &y[i]  (char, so no ×4)
  lbu  $t2, 0($t1)
  add  $t3, $s0, $a0       # &x[i]
  sb   $t2, 0($t3)
  beq  $t2, $zero, L2      # stop at '\0'
  addi $s0, $s0, 1
  j    L1
L2: lw   $s0, 0($sp)
  addi $sp, $sp, 4
  jr   $ra
```

Trap: Elements of a char array are 1 byte, so **do not multiply by 4 with sll**. This is the difference from an int array.

Lecture note `Week 4-2`
For a char array, do not multiply the index by 4 (sll). lbu reads just 1 byte. lbu is also I-type, so the offset is a constant — putting a register there gets 'zero points whether the answer is right or not'.

Addressing · 32-bit Constants · Translating and Linking

Slides: week3 p.49–67

§ 32-bit Constants: lui + ori

Most constants fit in a 16-bit immediate. When a 32-bit constant is occasionally needed:

- `lui rt, constant`: copies the 16-bit constant into the **upper 16 bits** of rt and clears the lower 16 bits to 0.
- Fill the lower 16 bits with **ori** (do not use addi, because of sign extension).

```
lui $s0, 61          # 0000 0000 0111 1101 0000 0000 0000 0000
ori $s0, $s0, 2304   # 0000 0000 0111 1101 0000 1001 0000 0000
```

§ Branch Addressing (PC-relative)

op	rs	rt	constant or address (16)

- beq/bne are I-format. Most branch targets are near the branch (loops, ifs).
- **Target address = (PC + 4) + offset × 4**
- The offset is the **difference in number of instructions (words)**, signed → range **$\pm 2^{15}$ words** (= $\pm 2^{17}$ bytes).
- If the target is too far, the assembler rewrites the branch by inverting the condition and using j:

```
beq $s0, $s1, L1      →      bne $s0, $s1, L2
                             j   L1
                        L2: …
```

Exam point: In Week 6 the professor explained it again on the board: offset = (target – (PC + 4)) / 4. E.g., PC 108, target 124 → (124 – 112)/4 = **3**. (The Week 4-2 preview was rough; the Week 6 explanation is the exact rule.)

Lecture note `Week 4-2`
If the branch target is too far to be represented in 16 bits, place an intermediate point and jump twice. Details on the final. (For the exact offset rule, see the Week 6 board notes.)

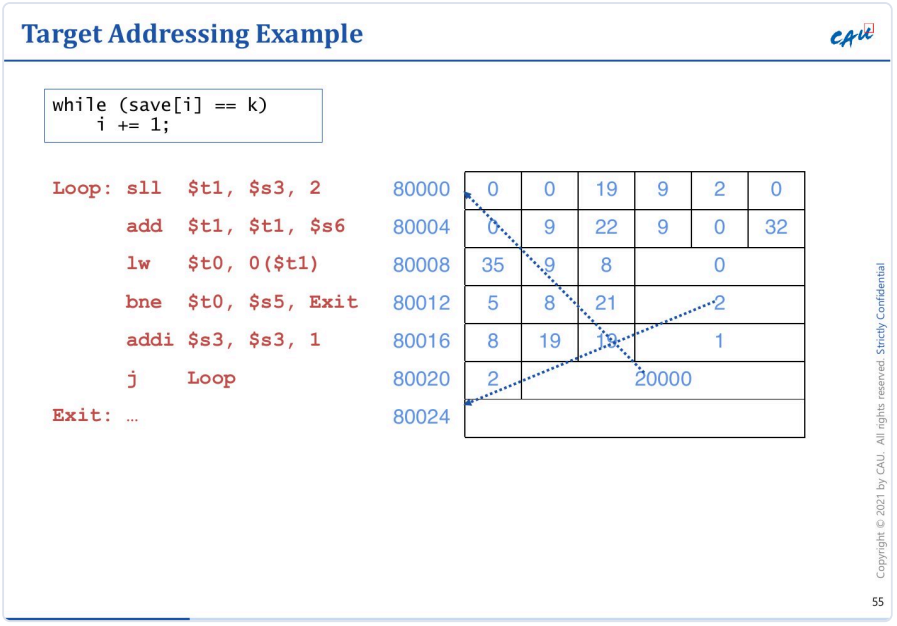
§ Jump Addressing (J-format)

op (6)	address (26)

- j, jal: the target can be anywhere in the text segment → encode the absolute address.
- (Pseudo) direct jump addressing: **Target = address × 4** (upper 4 bits come from the PC). Range 2^{26} words = 2^{28} bytes.
- `jr`: the full 32-bit address is in a register → R-format.

Slide example (Loop at address 80000):

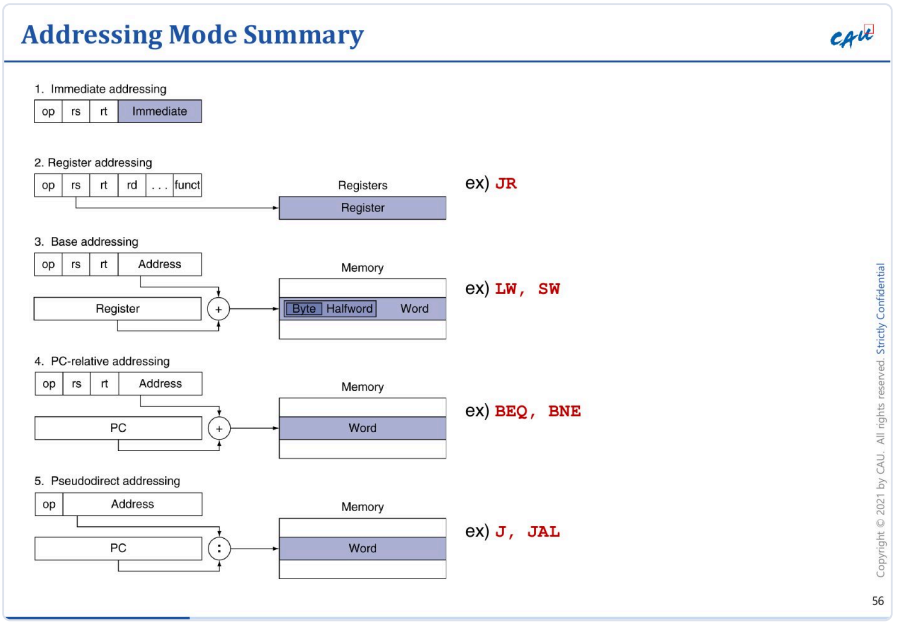
```
80000 Loop: sll  $t1,$s3,2     0  0 19  9 2  0
80004       add  $t1,$t1,$s6   0  9 22  9 0 32
80008       lw   $t0,0($t1)   35  9  8   0
80012       bne  $t0,$s5,Exit  5  8 21   2      → (80012+4) + 2×4 = 80024 = Exit
80016       addi $s3,$s3,1     8 19 19   1
80020       j    Loop          2   20000        → 20000 × 4 = 80000 = Loop
80024 Exit:
```



week3 p.56 · Target Addressing Example (loop at address 80000)

§ Addressing Mode Summary

1. Immediate (addi), 2. Register (add, jr), 3. Base/displacement (**lw, sw**), 4. PC-relative (**beq, bne**), 5. Pseudo-direct (**j, jal**).



week3 p.57 · Addressing Mode Summary

§ Pseudo-instructions · Translation · Linking

- **Pseudo-instruction**: "figments of the assembler's imagination" that the assembler turns into real instructions. `move $t0,$t1` → `add $t0,$zero,$t1`, `blt $t0,$t1,L` → `slt $at,$t0,$t1` + `bne $at,$zero,L`.
- Object module: header, text segment, static data, relocation info, symbol table, debug info.
- **Linker**: merges segments → resolves label addresses → patches location-dependent and external references → executable.
- **Loader**: reads the header → creates the virtual address space → copies text and data → puts arguments on the stack → initializes registers ($sp, $fp, $gp) → jumps to the startup routine.
- **Dynamic linking**: link/load a library procedure only when it is called. Requires relocatable code; avoids image bloat; automatically picks up new library versions.
- **Memory layout**: Text (code) → Static data (globals, $gp) → Dynamic data (heap, malloc) → Stack (automatic storage).

---

§ Fallacies and Design Principles Summary

- Fallacy: powerful instructions = higher performance (complex instructions are hard to implement and may slow down even the simple ones).
- Fallacy: use assembly code for high performance (modern compilers do better).
- x86 never changes its instruction set, it only keeps adding (backward compatibility).
- **The four design principles**: 1. Simplicity favors regularity, 2. Smaller is faster, 3. Make the common case fast, 4. Good design demands good compromises.

U3 · week4-수업.pdf p.6–21 · week5-수업.pdf

Ch.3 Arithmetic for Computers (Integer · Floating Point)

Multiplication and division come as "given the circuit, show the process". The professor said "there may be small changes in the circuit, so look carefully". On the midterm the sizes are only product/multiplicand 8 bits and multiplier 4 bits. In floating point, getting even one field's bit count wrong gets zero points.

Addition · Subtraction · Overflow

Slides: week4 p.6–13

§ Integer Addition and Subtraction

- Addition works like decimal: add digit by digit, and if a column reaches 2 or more, carry to the next column.
- Example: 7 + 6 = `0111 + 0110 = 1101` (13).
- **Subtraction = adding the negative**: 7 − 6 = 7 + (−6). Make −6 using two's complement (invert + 1).

§ Detecting Overflow

Overflow occurs if the result is out of range (e.g., 32 bits).

Operation	Overflow condition
Positive + negative	**None**
Positive + positive	Result sign bit is **1**
Negative + negative	Result sign bit is **0**
Positive − positive, negative − negative	None
Negative − positive	Result sign is **0**
Positive − negative	Result sign is **1**

Why: in two's complement the first bit is the sign. Adding two positives cannot give a negative, so a sign bit of 1 means overflow. Modern CPUs handle overflow as an **exception**.

> **Lecture note** `Week 4-2`
> Positive + negative never overflows. Looking at the first bit (sign) alone is enough to decide. Modern CPUs handle overflow as an exception to protect the circuit.

Multiplication Hardware

Slides: week4 p.14–18 · week5 p.5–8, 14–15

§ How Multiplication Works

Same as long multiplication on paper: look at each digit of the multiplier; if it is 1, add the multiplicand (shifted by that position); if it is 0, add nothing.

```
  multiplicand     1000
× multiplier     × 1001
                   1000
                  0000
                 0000
                1000
  product       1001000      (8 × 9 = 72)
```

n-bit × n-bit → the product is up to **2n bits** long.

§ The Original Multiplier (first version)

Components: **Multiplicand 64-bit (Shift left)**, **Multiplier 32-bit (Shift right)**, **64-bit ALU**, **Product 64-bit**, Control test. (Scaled down to 8-bit / 4-bit / 8-bit on the midterm.)

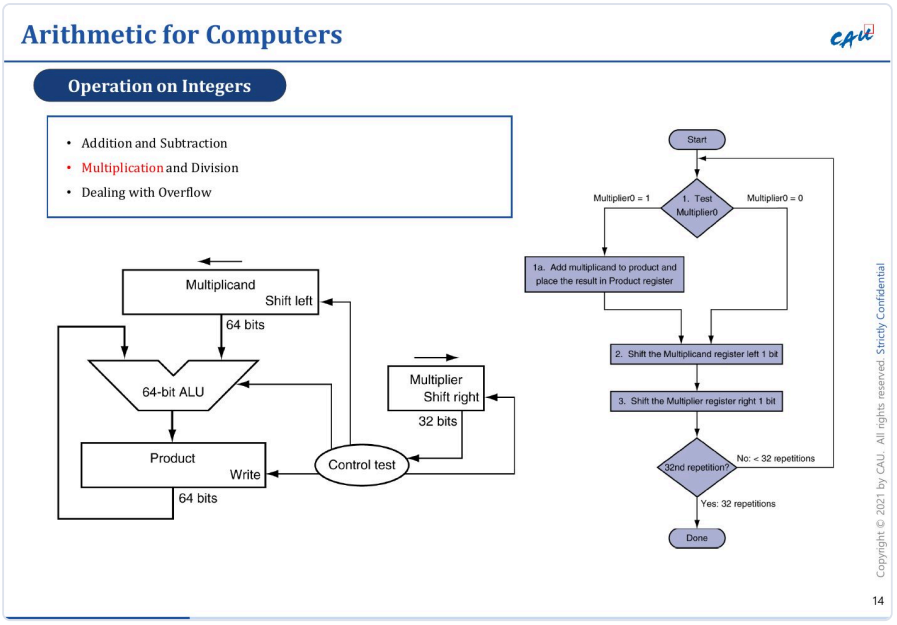
Algorithm:

1. Initialize **Product = 0**.
2. **Test the LSB** of the Multiplier: if 1, `Product = Product + Multiplicand` (send an add signal to the ALU); if 0, do nothing.
3. **Every iteration**, regardless of the result: shift the Multiplicand **left** and the Multiplier **right**.
4. Done after 32 repetitions (4 at midterm size). (As the professor explained: it ends when the multiplier becomes 0.)

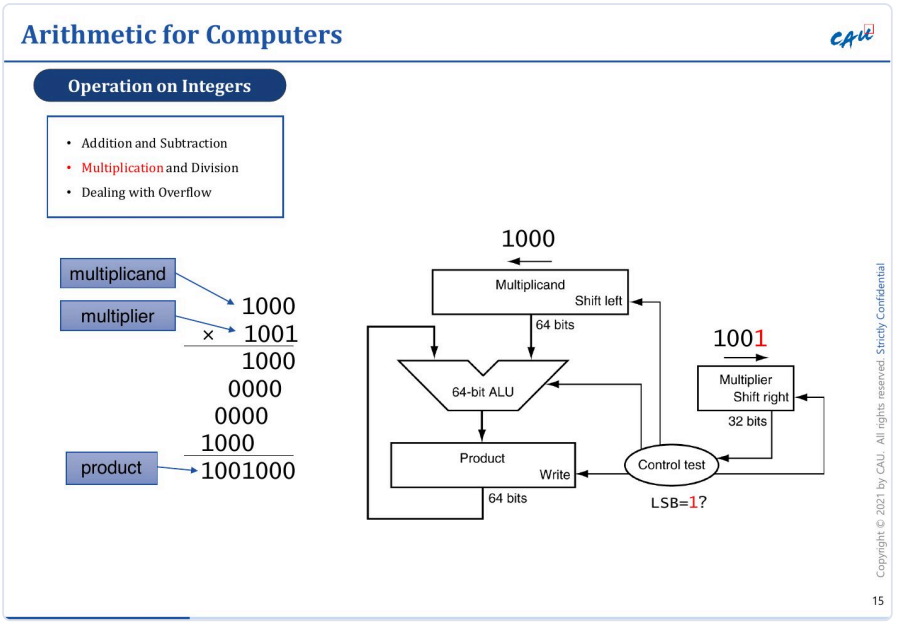
1000 × 1001:

Iteration	Step	Multiplier	Multiplicand	Product
0	Initial values	1001	0000 1000	0000 0000
1	LSB=1 → Prod += Mcand	1001	0000 1000	0000 1000
1	Mcand <<1, Mplier >>1	0100	0001 0000	0000 1000
2	LSB=0 → no add, shift	0010	0010 0000	0000 1000
3	LSB=0 → no add, shift	0001	0100 0000	0000 1000
4	LSB=1 → Prod += Mcand	0001	0100 0000	0100 1000
4	shift	0000	1000 0000	**0100 1000** (= 72)

> **Lecture note** Week 4-2 · Week 5-1
> On the midterm it only appears with an 8-bit product and multiplicand and a 4-bit multiplier. Regardless of the LSB result, **shift twice every iteration** (multiplier right, multiplicand left).



week4 p.15 · The original multiplier and its flowchart



week4 p.16 · 1000 × 1001: testing the LSB

§ The Refined Multiplier (refined version)

Components: **Multiplicand 32-bit (not shifted)**, **32-bit ALU**, **Product 64-bit (Shift right)**. There is no separate Multiplier register.

Algorithm:

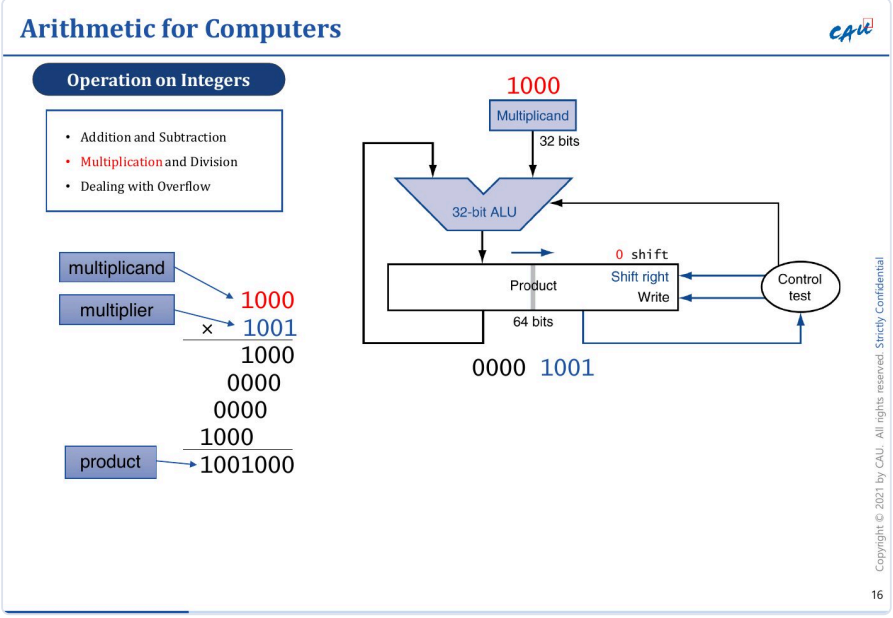
1. Put the **multiplier in the lower half** of Product and 0 in the upper half.
2. **Test the LSB** of Product: if 1, **add the multiplicand to the upper half** of Product.
3. Every iteration: **shift the whole Product right**.
4. Termination: **number of shifts = number of bits in the multiplier** (shift counter).

Trap: The termination condition counts the bits of the **multiplier**, not the multiplicand (the professor: "be extremely careful it is not the multiplicand").

1000 × 1001:

Iteration	Step	Product
0	Initial value (lower half = multiplier)	0000 1001
1	LSB=1 → upper half += 1000	1000 1001
1	>> 1	0100 0100
2	LSB=0, >> 1	0010 0010
3	LSB=0, >> 1	0001 0001
4	LSB=1 → upper half += 1000	1001 0001
4	>> 1	**0100 1000** (= 72)

Lecture note `Week 5-1`
'I will actually provide you a circuit and ask you what to do for the operation ... there will be minor tweaks within the circuit'. It terminates when the number of shifts = the number of bits of the **multiplier** (not the multiplicand).



week4 p.17 · The refined multiplier (Product shifts right)

§ Class Examples 1 and 2 (Week 5)

Example 1: 1010 × 0010 (original multiplier) → 10 × 2 = 20

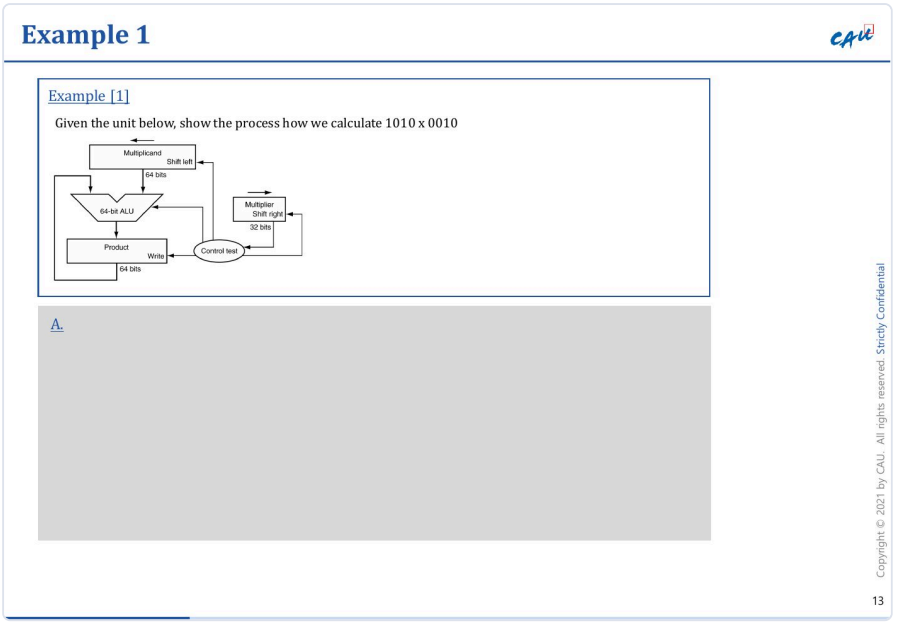
Iteration	LSB	Action	Multiplier (after shift)	Multiplicand (after shift)	Product
0		Initial values	0010	0000 1010	0000 0000
1	0	No add	0001	0001 0100	0000 0000
2	1	Prod += 0001 0100	0000	0010 1000	0001 0100
3	0	No add	0000	0101 0000	0001 0100
4	0	No add	0000	1010 0000	**0001 0100** (= 20)

Example 2: 1010 × 1100 (refined multiplier) → 10 × 12 = 120

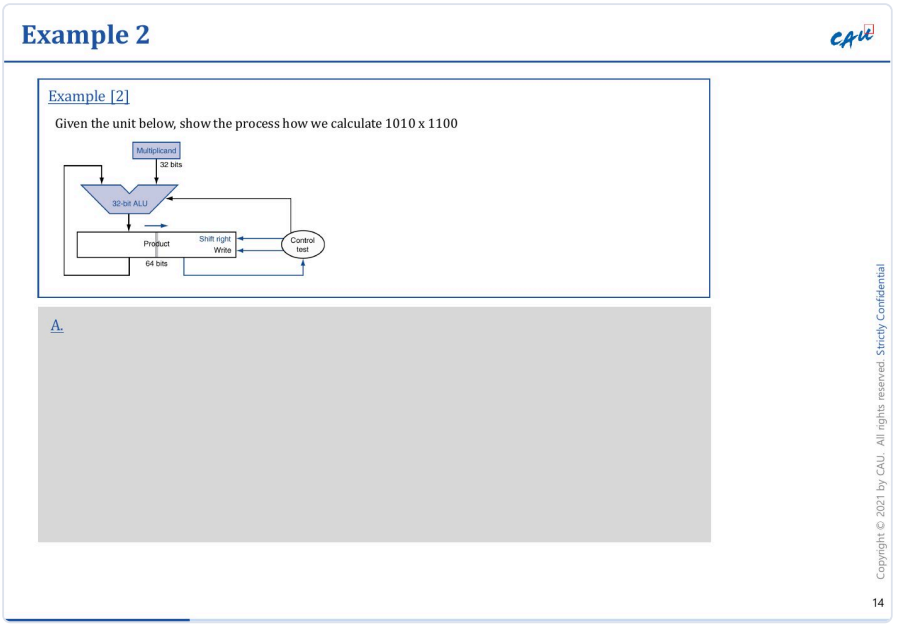
Iteration	LSB	Action	Product (after action)	Product (after >>1)
0		Initial value	0000 1100	
1	0	No add	0000 1100	0000 0110
2	0	No add	0000 0110	0000 0011
3	1	Upper half += 1010	1010 0011	0101 0001
4	1	Upper half += 1010	1111 0001	**0111 1000** (= 120)

Lecture note `Week 5-1`

Exam format preview: given an instruction and a description, fill in the boxes or write the correct MIPS expression. For multiplication and division, a circuit is given and you are asked for the process.



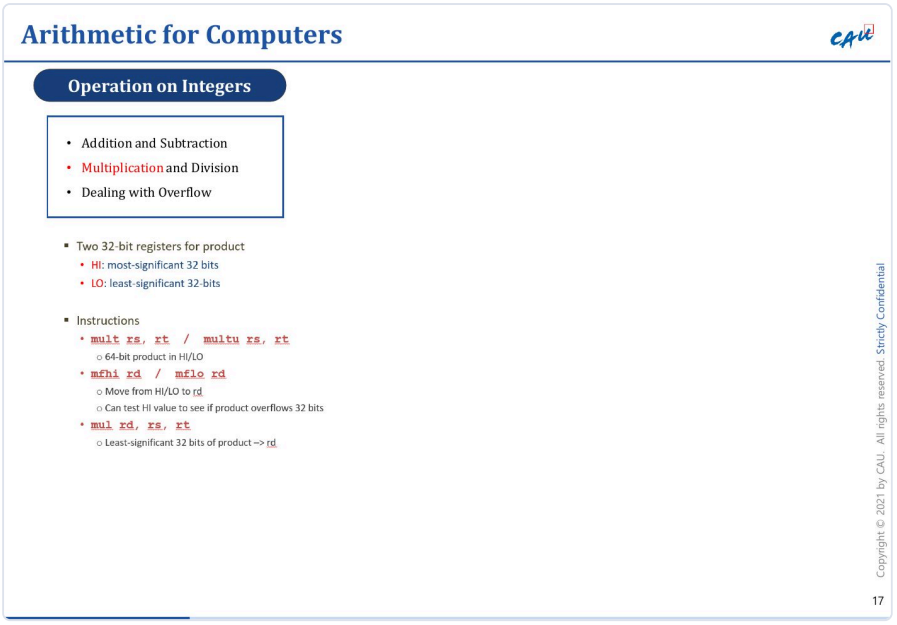
week5 p.14 · Example 1 (original multiplier)



week5 p.15 · Example 2 (refined multiplier)

§ MIPS Multiplication Instructions

- Two registers hold the 64-bit product: **HI** (upper 32 bits) and **LO** (lower 32 bits).
- `mult rs, rt` / `multu rs, rt`: 64-bit product into HI/LO.
- `mfhi rd` / `mflo rd`: move HI/LO to rd. Checking HI tells you whether the product overflows 32 bits.
- `mul rd, rs, rt`: only the lower 32 bits of the product go to rd.



week4 p.18 · HI/LO, mult, mfhi, mflo, mul

Division Hardware

Slides: week4 p.19–21 · week5 p.9–13, 16

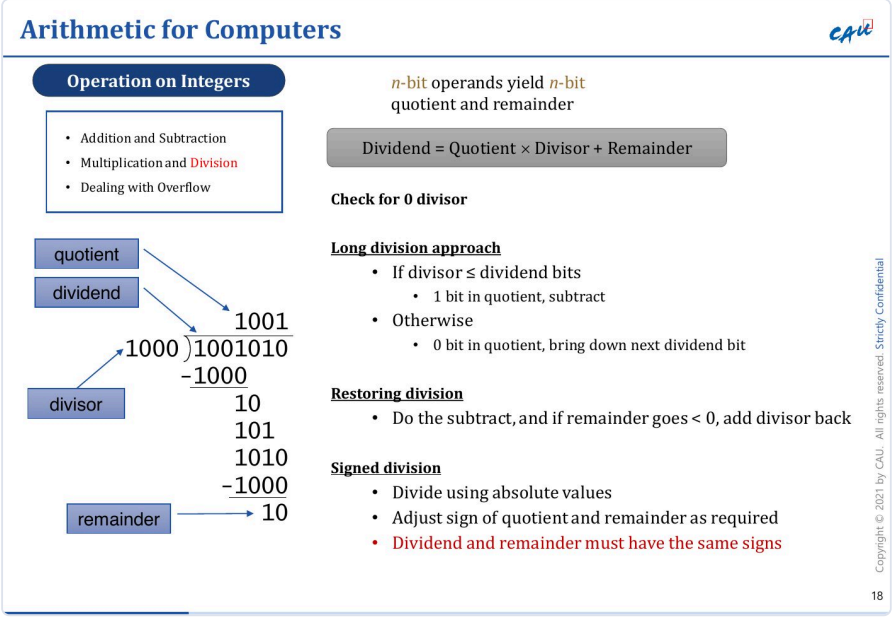
§ How Division Works

`Dividend = Quotient × Divisor + Remainder`. n-bit operands yield an n-bit quotient and remainder.

- **Check for a 0 divisor first** (slide: "Always check for 0 divisor!!").
- Long division approach: if divisor ≤ the dividend bits currently being examined, put 1 in the quotient and subtract; otherwise put 0 in the quotient and bring down the next dividend bit.
- **Restoring division**: do the subtract, and if the remainder goes **negative (< 0), add the divisor back** to restore it.
- Signed division: divide using absolute values, then adjust the signs of the quotient and remainder as required. **The dividend and remainder must have the same sign.**

Slide example: 1001010 ÷ 1000 = quotient **1001**, remainder **10** (74 ÷ 8 = 9 remainder 2).

> **Lecture note** `Week 4-2 · Week 5-1`
> Explained with the decimal division 728 ÷ 8: subtract first, and if it goes negative, add it back (plus back) to restore it, then move to the next digit.



week4 p.19 · How division works and restoring division

§ The Divider (first version)

Components: **Divisor 64-bit (Shift right, initially in the left half)**, **64-bit ALU**, **Remainder 64-bit (initially the dividend)**, **Quotient 32-bit (Shift left)**.

Algorithm (repeat n+1 times):

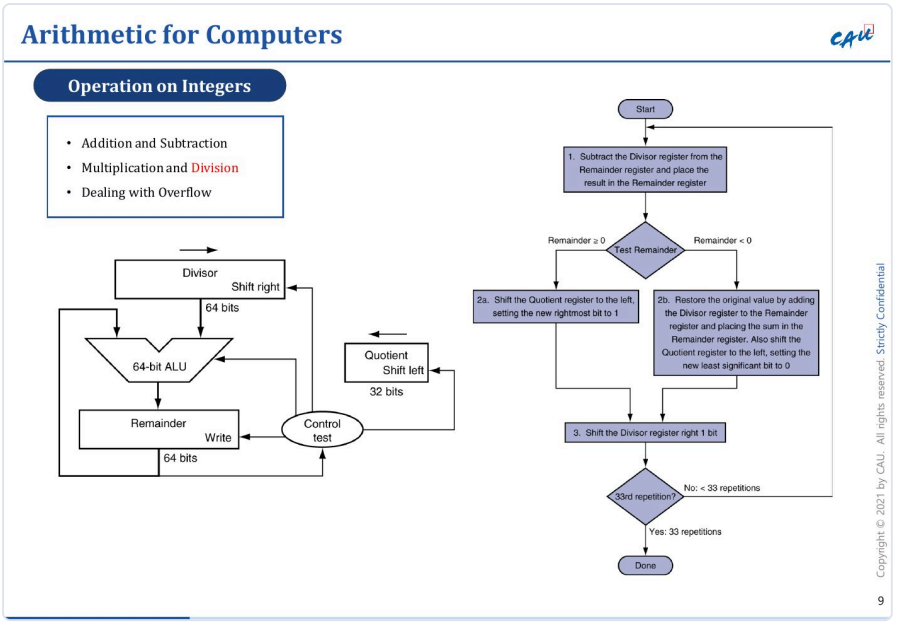
1. `Remainder = Remainder - Divisor`
2. Test the result:
   - **≥ 0**: shift Quotient left, new LSB = **1**

- **< 0**: **restore** with `Remainder = Remainder + Divisor`, shift Quotient left, new LSB = **0**

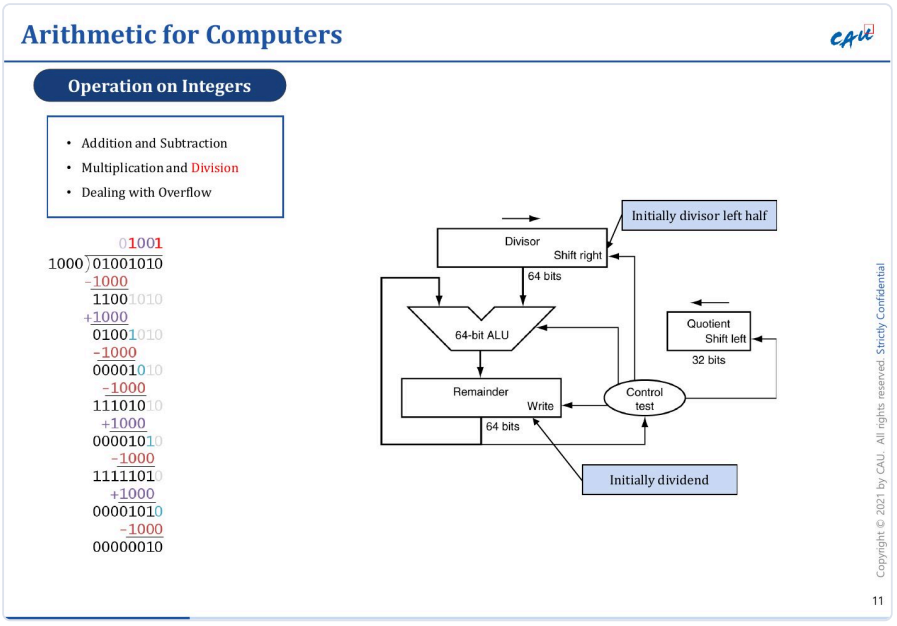
3. Shift Divisor **right**.

Trap: Whether the result is negative is decided by its **first bit (sign bit)** (two's complement). If it is negative, you must add the divisor back to restore the original value before moving on to the next bit.

Lecture note `Week 5-1`
Whether it is negative is known from the first bit of the two's complement. If it is negative you cannot append a quotient bit, and the same value has to be joined with the next digit, so you must add back to restore the original value.



week5 p.10 · The divider and its flowchart



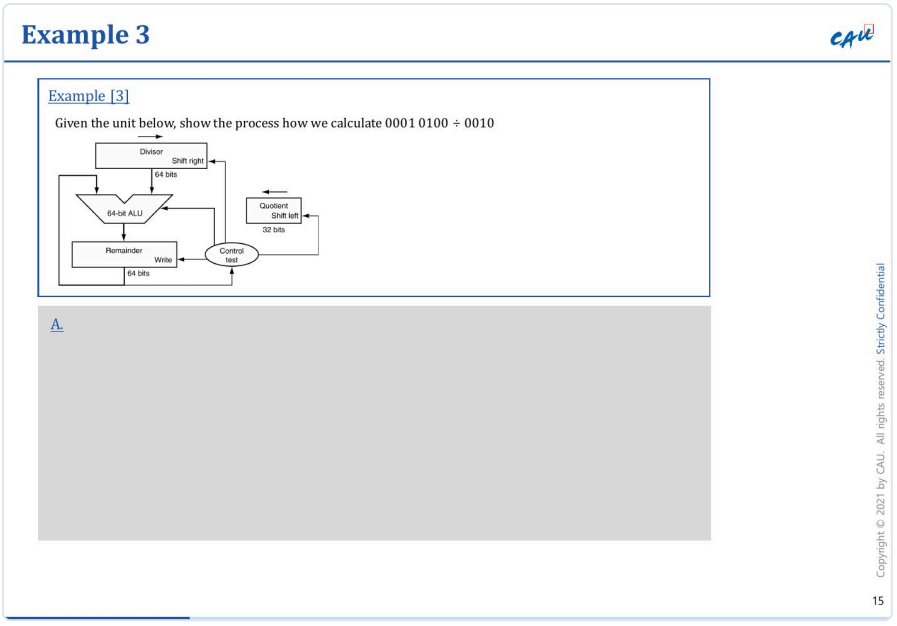
week5 p.12 · 01001010 ÷ 1000: subtract and restore steps

§ Class Example 3: 0001 0100 ÷ 0010

20 ÷ 2 = 10 remainder 0. A 4-bit division, so **5** iterations. Initial Divisor = 0010 in the left half → `0010 0000`.

Iteration	Rem – Div	Decision	Quotient	Divisor (after >>1)	Remainder
0		Initial values	0000	0010 0000	0001 0100
1	1111 0100	< 0 → restore, 0	0000	0001 0000	0001 0100
2	0000 0100	≥ 0 → 1	0001	0000 1000	0000 0100
3	1111 1100	< 0 → restore, 0	0010	0000 0100	0000 0100
4	0000 0000	≥ 0 → 1	0101	0000 0010	0000 0000
5	1111 1110	< 0 → restore, 0	**1010**	0000 0001	**0000 0000**

Result: Quotient = 1010 (10), Remainder = 0.



week5 p.16 · Example 3 (divider)

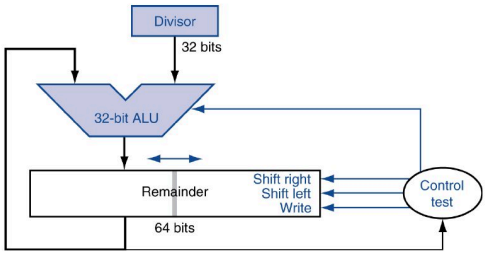
§ The Refined Divider

Slide week5 p.13: **Divisor 32-bit (not shifted)**, **32-bit ALU**, **Remainder 64-bit (Shift right / Shift left / Write)**. The quotient accumulates in the lower half of the Remainder register and the remainder stays in the upper half (same idea as the refined multiplier: merge into one register).

Arithmetic for Computers

Operation on Integers

- Addition and Subtraction
- Multiplication and Division
- Dealing with Overflow



Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

week5 p.13 · The refined divider

IEEE 754 Floating-Point Representation

Slides: week5 p.17–26

§ The IEEE 754 Format

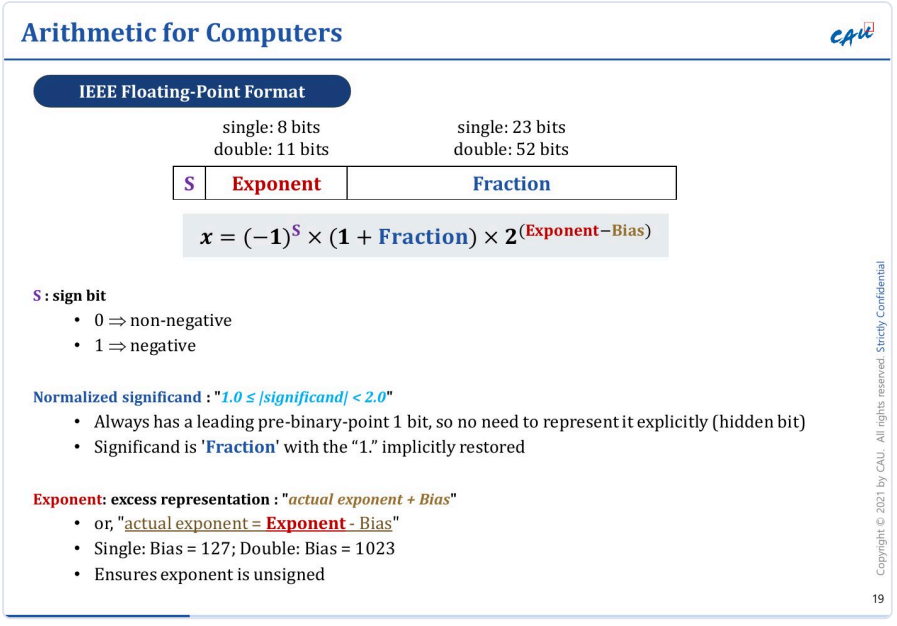
	S	Exponent	Fraction	Bias
Single (float)	1	**8 bits**	**23 bits**	**127**
Double (double)	1	**11 bits**	**52 bits**	**1023**

`x = (-1)^S × (1 + Fraction) × 2^(Exponent - Bias)`

- **S**: 0 = non-negative, 1 = negative.
- **Normalized significand**: 1.0 ≤ |significand| < 2.0. The leading "1." is always there, so it is **not stored (hidden bit)**.
- **The exponent uses excess (biased) representation**: stored value = actual exponent + Bias. So the stored exponent is always unsigned.

Trap: If you get any field width wrong (1/8/23, 1/11/52), the other fields shift too and the **whole answer gets zero points** (the professor, Week 5).

Lecture note `Week 5-1`
If you miss even one field's bit count, the fields get mixed up and you get **zero points**. You must write out the whole representation.



week5 p.20 · IEEE 754 format

§ Conversion Examples

−0.75: 0.75 = 0.11_2 = $\mathbf{1.1_2 \times 2^{-1}}$ → S = 1, Exponent = −1 + 127 = 126 = `01111110`, Fraction = `1000…0`

```
1 01111110 10000000000000000000000   = 0xBF400000
```

Double: Exponent = −1 + 1023 = 1022 = `01111111110`, Fraction = `1000…0` (52 bits).

0.625: 0.101_2 = **$1.01_2 \times 2^{-1}$** → S = 0, E = 126, F = `01000…0` → `0 01111110 01000000000000000000000` = 0x3F200000

−3.75: 11.11_2 = **$1.111_2 \times 2^{1}$** → S = 1, E = 1 + 127 = 128 = `10000000`, F = `111000…0` → `1 10000000 11100000000000000000000` = 0xC0700000

Steps: ① determine the sign ② convert to a binary fraction ③ normalize to 1.xxx × 2^e ④ e + bias ⑤ put the bits after the binary point into the fraction (rest 0).

§ Range · Special Values · Precision

In single precision, exponents `00000000` and `11111111` are **reserved**:

Exponent	Fraction	Meaning
0	0	**0**
0	≠ 0	**Denormal** (hidden bit = 0, `(0 + F) × 2^(-bias)`)
255	0	**±∞**
255	≠ 0	**NaN** (undefined result such as 0.0/0.0)

- Single smallest (normalized): exponent 1 → 1 − 127 = −126, $\pm 1.0 \times 2^{-126} \approx \pm 1.2 \times 10^{-38}$
- Single largest: exponent 254 → +127, $\approx \pm 2.0 \times 2^{127} \approx \pm 3.4 \times 10^{38}$
- Double: −1022 to +1023, $\approx \pm 2.2 \times 10^{-308}$ to $\pm 1.8 \times 10^{308}$
- Precision: single $\approx 2^{-23}$ → about **6 decimal digits**; double $\approx 2^{-52}$ → about **16 digits**.
- Denormals represent values smaller than the normalized numbers, allowing **gradual underflow**.

Floating-Point Addition · Multiplication · FP Instructions

Slides: week5 p.27–31

§ FP Addition in 4 Steps

1. **Align**: shift the number with the smaller exponent to match the larger exponent.
2. **Add significands**.
3. **Normalize** the result and check for over/underflow.
4. **Round**, and renormalize if necessary.

Decimal example: $9.999\times10^{1} + 1.610\times10^{-1} \rightarrow 9.999\times10^{1} + 0.016\times10^{1} \rightarrow 10.015\times10^{1} \rightarrow 1.0015\times10^{2} \rightarrow$ **1.002×10^{2}**

Binary example: $0.5 + (-0.4375) = 1.000_2\times2^{-1} + (-1.110_2\times2^{-2})$

```
1. 1.000×2⁻¹ + (-0.111×2⁻¹)
2. 0.001×2⁻¹
3. 1.000×2⁻⁴  (no over/underflow)
4. 1.000×2⁻⁴ = 0.0625
```

§ FP Multiplication in 5 Steps

1. **Add exponents** (for biased exponents, subtract the bias once from the sum).
2. **Multiply significands**.
3. **Normalize**, check for over/underflow.
4. **Round**, renormalize if necessary.
5. **Determine the sign** (from the operands' signs).

Binary example: $0.5 \times (-0.4375) = 1.000_2\times2^{-1} \times (-1.110_2\times2^{-2})$

```
1. exponent: -1 + -2 = -3   (biased: (-1+127) + (-2+127) - 127 = -3 + 127)
2. 1.000 × 1.110 = 1.110  → 1.110×2⁻³
3. already normalized, no over/underflow
4. rounding leaves it unchanged
5. +ve × -ve → -ve :  -1.110×2⁻³ = -0.21875
```

Trap: Adding two biased exponents counts the bias twice, so you must **subtract 127 once**.

§ FP Hardware and Instructions

- Usually dedicated FP hardware (multi-cycle): addition, subtraction, multiplication, division, reciprocal, square root, FP ↔ integer conversion.
- **Separate FP registers**: 32 single-precision `$f0-$f31`; doubles use pairs `$f0/$f1`, `$f2/$f3`, ...
- FP instructions operate only on FP registers (integer operations are not applied to FP data) → more registers with minimal impact on code size.

U4 · week6-수업.pdf

Ch.4 The Processor: Datapath and Control (Simplified Version)

In Week 6-2 the professor said "today's goal is to completely understand this one slide (p.3, the simplified CPU)". The key is which wires the data flows through for each instruction, and whether each MUX gets 0 or 1. After reading the concepts, use the '수업 따라가기' (Follow the Lecture) tab to see the professor's explanation scene by scene, and the '데이터패스 놀이터' (Datapath Playground) to check the path for each instruction.

Processor Overview · Instruction Execution Steps

Slides: week6 p.2–7

§ Two Implementations and the Instruction Subset

- CPU performance factors: Instruction Count, **CPI and Cycle Time** (this unit covers the implementation, which determines CPI and cycle time).
- Two MIPS implementations: **a simplified version (midterm)** and a more realistic pipelined version (final).
- Instructions covered: memory reference `lw, sw` · arithmetic/logical `add, sub, and, or` · control transfer `beq, j` (p.4 also includes `slt`).

Exam point: The professor said "on the midterm you must use only these instructions. Using any other instruction gets zero points." In class he said "seven instructions", but the slide list is lw, sw, add, sub, and, or, beq, j. The simplified datapath actually covered R-type, lw, sw and beq.

Lecture note `Week 6-1`
'before midterm, you will actually deal with the simplified version of the CPU, and for your final, you will deal with the pipeline version'. On the midterm use only the instructions defined in class; using any other gets zero points.

§ Instruction Execution Steps

1. **PC → instruction memory**, fetch the instruction.
2. **Register numbers → register file**, read the registers.
3. Depending on the instruction class:
   - Use the **ALU** to calculate: the arithmetic result / the **memory address** for load/store / the **branch target address**
   - Access **data memory** for load/store
   - **PC ← branch target address or PC + 4**

An instruction is 1 word = **32 bits = 4 bytes** → to go to the next line, **PC + 4**.

Trap: In the Week 6-1 recording the professor said "word = 32 bytes"; that was a slip of the tongue. It is 32 **bits** = 4 bytes.

§ CPU Overview and Why MUXes Are Needed

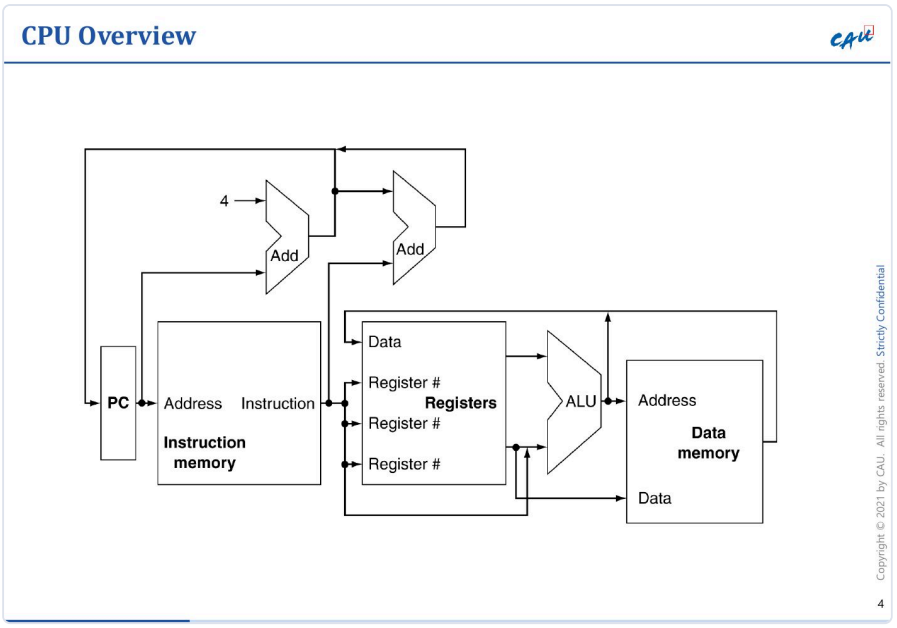
p.5 overview: PC → Instruction memory → Registers → ALU → Data memory, plus the PC+4 adder and the branch adder.

p.6: **"Can't just join wires together: Use multiplexers"** — wires from different sources cannot simply be joined, so a selector is needed in 3 places:

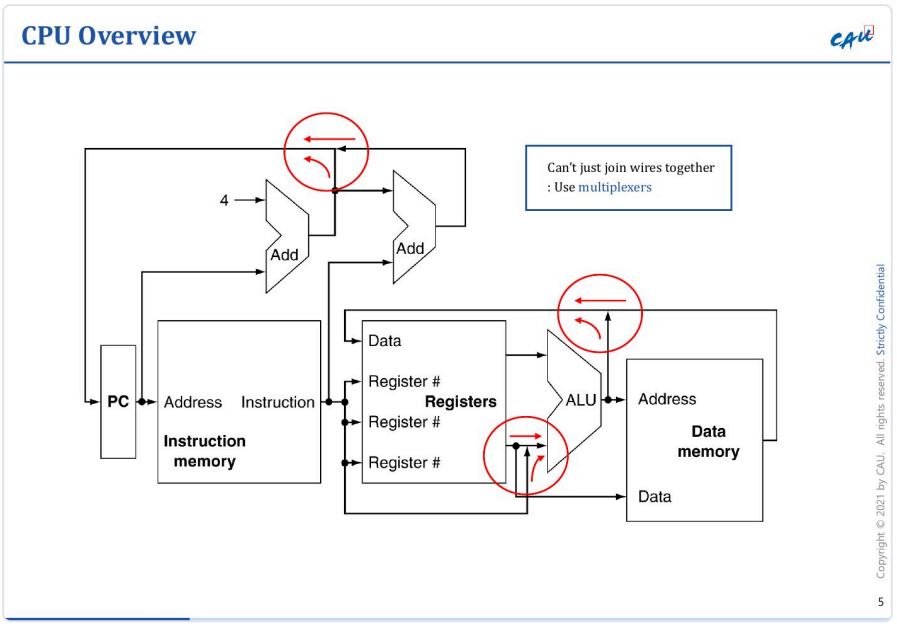
1. **PC input**: PC + 4 or the branch target
2. **Second ALU input**: a register value or the instruction's 16-bit constant (sign-extended)
3. **Register write data**: the ALU result or the value read from data memory

p.7: a **Control** unit is added, which generates the MUX select signals, RegWrite, MemRead/MemWrite, the ALU operation, and Branch.

The professor explained: for R-format, the ALU result is the value we want; but for lw the ALU result is **only an address**, and what we actually want is the memory value at that address. That is why the write-data MUX is needed.

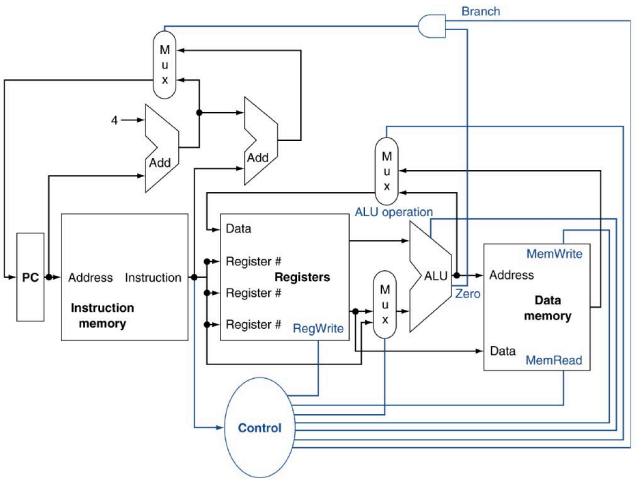


week6 p.5 · CPU Overview



week6 p.6 · Can't just join wires together: the 3 places that need a MUX

CPU Overview: Control



Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

6

week6 p.7 · CPU Overview: Control

Logic Design Basics · Clocking

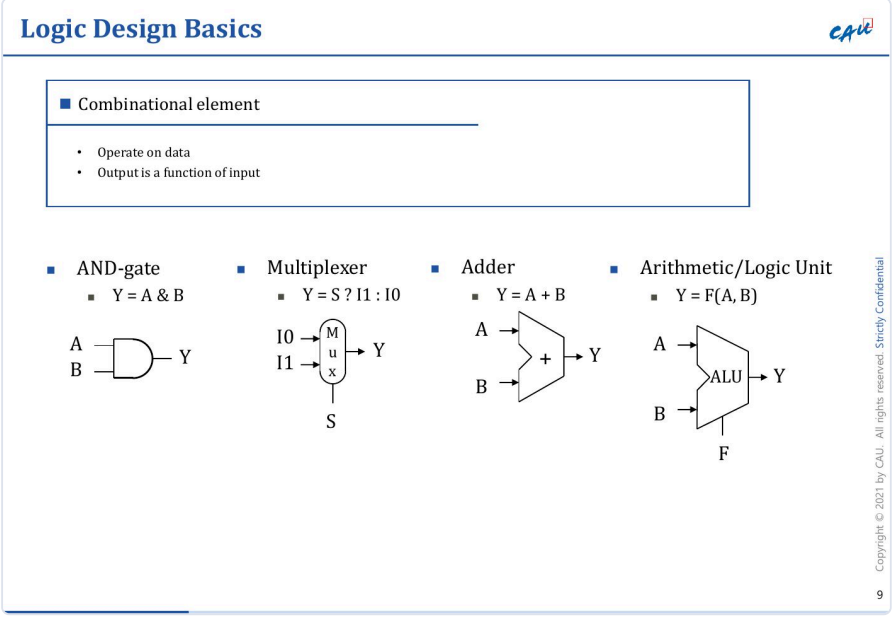
Slides: week6 p.8–15

§ Information Encoding and Combinational Elements

- Information encoded in binary: low voltage = 0, high voltage = 1. **One wire per bit**; multi-bit data is encoded on multi-wire buses (32 bits = 32 wires).
- **Combinational element**: operates on data. **The output is a function of the input only** (no memory).

Element	Expression
AND gate	Y = A & B
Multiplexer	**Y = S ? I1 : I0**
Adder	Y = A + B
ALU	Y = F(A, B), F = the function to perform

The ALU is an extension of the adder. For R-format, F is determined by the lowest 6 bits of the instruction (**funct**).



week6 p.10 · AND, MUX, Adder, ALU

§ How to Read a Multiplexer

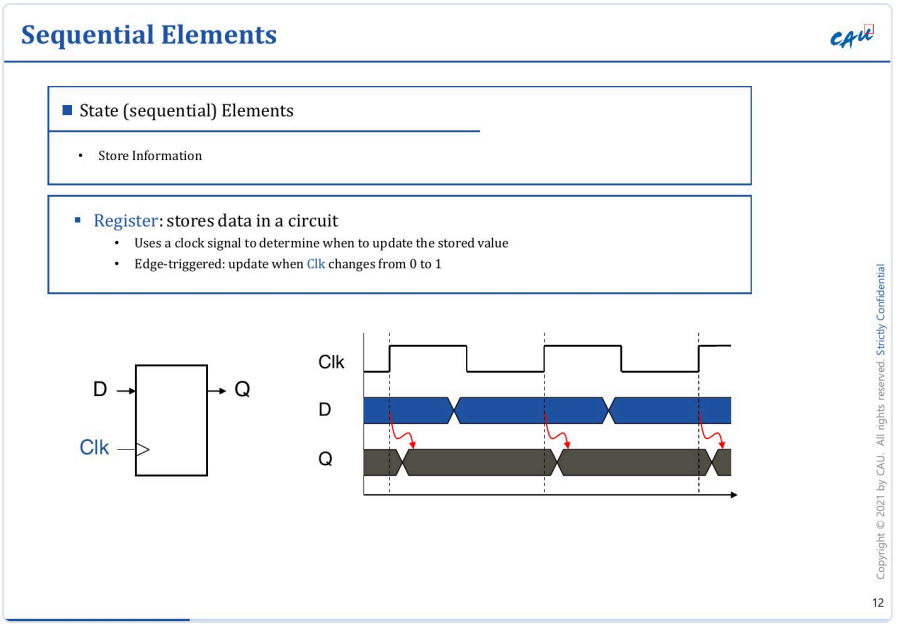
- Each input wire is numbered (I0, I1).
- **Select line S = 0 → I0, S = 1 → I1**.
- On the exam, **the figure shows which wire is 0 and which is 1**. The professor wrote the 0s and 1s on the board when explaining.

> Exam point: Decide the signal value by **the number written on the MUX**, not by "top/bottom". If the numbers are swapped, the signal is reversed too (Week 6-2: "be extremely careful on what is actually written on the multiplexer").

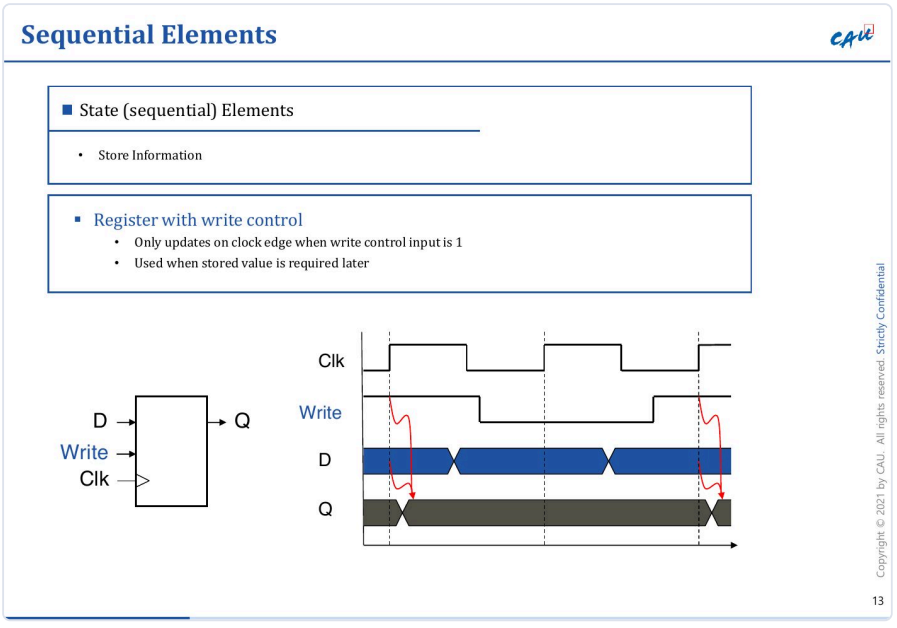
§ Sequential Elements: Registers

- **State (sequential) element**: **stores** information.
- **Register**: stores data in a circuit. Uses a **clock** signal to determine when to update the stored value.
- **Edge-triggered**: updates when Clk changes **from 0 to 1 (rising edge)**.
- **Register with write control**: only updates on a clock edge **when the write control input is 1**. Used when the stored value is required later. (The register file's RegWrite is this.)

Trap: In Week 6-1 the professor said "0 to 1 or 1 to 0", but the slide and the Week 6-2 explanation say **0 → 1**. The exam follows the slide.



week6 p.13 · Edge-triggered register



week6 p.14 · Register with write control

§ Clocking Methodology

- Combinational logic transforms data during clock cycles (**between clock edges**).

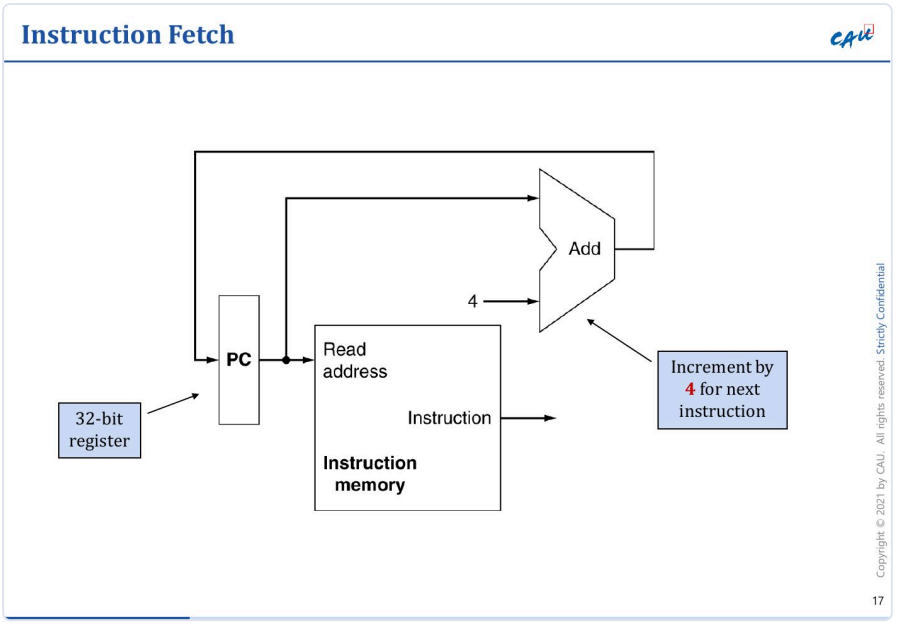
- Input comes from state elements; output goes to a state element.
- **The longest delay determines the clock period**.

Datapath Elements

Slides: week6 p.16–24

§ Building a Datapath · Instruction Fetch

- **Datapath**: the elements that process data and addresses in the CPU (registers, ALUs, MUXes, memories, ...). We build it incrementally for a simple subset of instructions.
- **Instruction fetch**: the **PC (a 32-bit register)** feeds the read address of the instruction memory → the instruction comes out. At the same time the PC goes into an **Add** unit, **+4** → the next instruction address goes back into the PC.



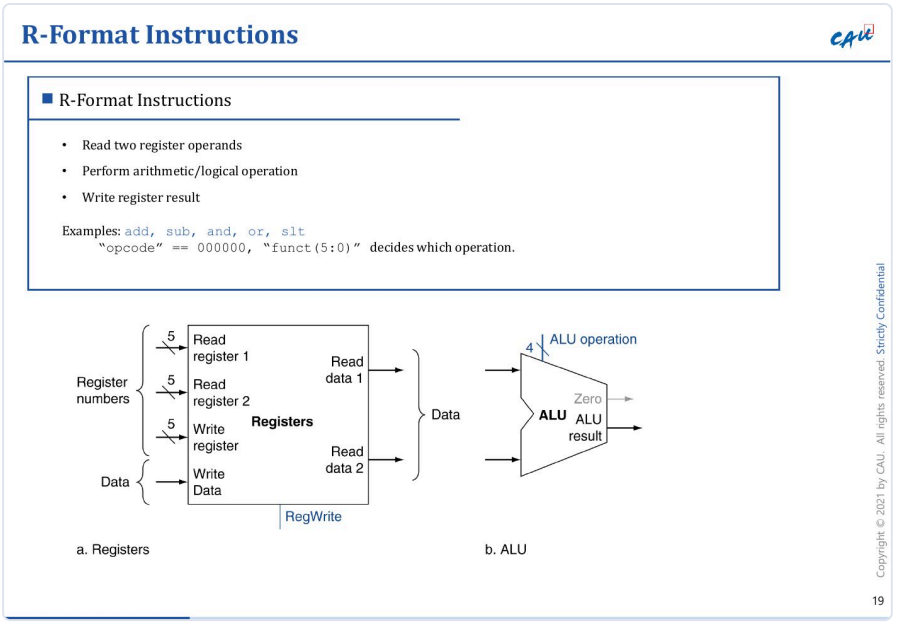
week6 p.18 · Instruction Fetch

§ The R-format Datapath

R-format (add, sub, and, or, slt): opcode = 000000, **funct (5:0)** selects the operation.

1. **Read two** register operands (Read register 1 = rs, Read register 2 = rt, 5 bits each)
2. Perform the arithmetic/logical operation (ALU, ALU operation 4 bits)
3. **Write** the result to a register (Write register = rd, Write data, **RegWrite = 1**)

Example `add $t0, $s1, $s2` → op 0 | rs 17 | rt 18 | rd 8 | shamt 0 | funct 32.



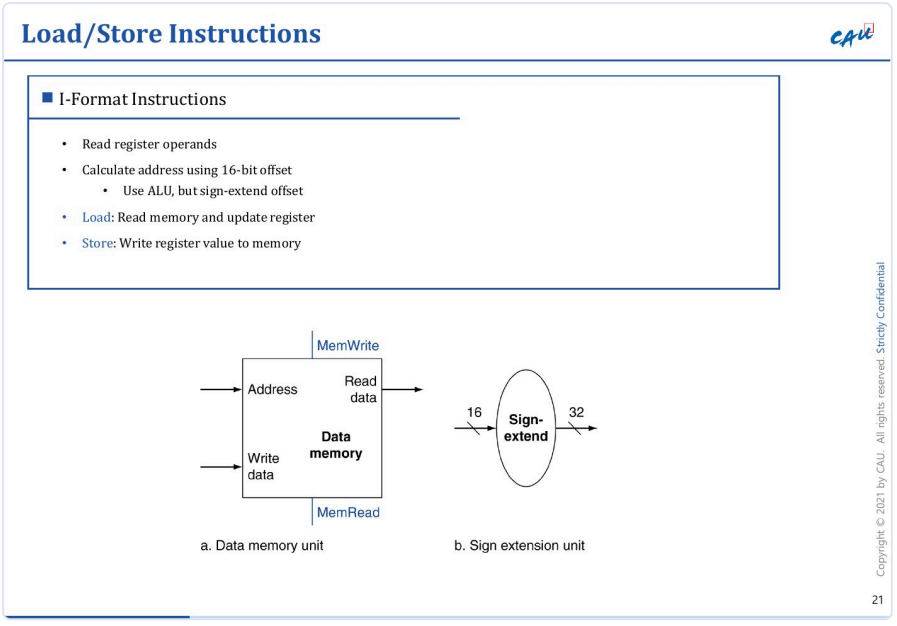
week6 p.20 · Registers and ALU

§ The Load/Store Datapath

1. Read the register operand (rs = base)
2. Calculate the address using the 16-bit offset: use the ALU, but **sign-extend the offset (16 → 32)**
3. **Load**: read memory and update the register (**MemRead = 1, RegWrite = 1**)
4. **Store**: write the register value to memory (**MemWrite = 1**)

Sign extension: fill the upper 16 bits with the constant's sign bit (0 for positive, 1 for negative).

Trap: At 23:18 in Week 6-1, while explaining lw, the professor said "turn on the memory write field"; that was a slip. lw uses **MemRead** and RegWrite; MemWrite is for sw. He corrected it right after (24:50).



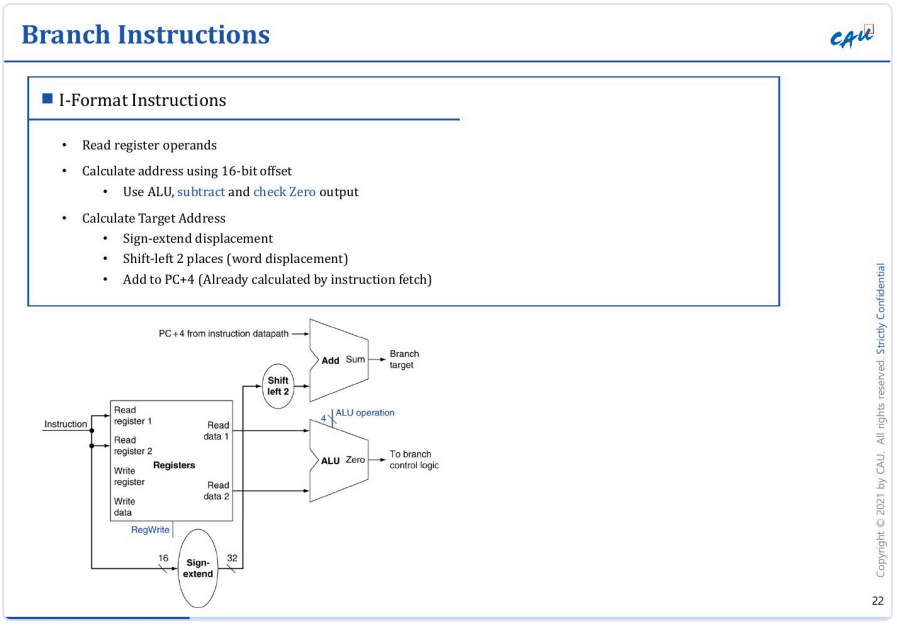
week6 p.22 · Data memory and Sign-extend

§ The Branch Datapath

beq:

1. Read the register operands (rs, rt)
2. **Subtract** with the ALU → check the **Zero output** (Zero = 1 if equal)
3. Calculate the target address:
   - **Sign-extend** the displacement (16 bits)
   - **Shift left 2** (word displacement → bytes, ×4)
   - Add to **PC + 4** (PC + 4 was already calculated during instruction fetch)

> Exam point: The branch constant is not an address but **how many lines (instructions) away from (PC + 4)** the target is. It is given in lines rather than bytes because a value not divisible by 4 would cause a serious error.



week6 p.23 · Branch datapath

§ Composing the Elements: The Four MUXes

- The simplified datapath executes **one instruction in one clock cycle**.
- Each element can do only one function at a time → **separate instruction memory and data memory**.
- A MUX wherever there are two sources:
  - ALU input: register vs. the instruction's immediate → **ALUSrc**
  - Register write data: ALU output vs. data memory → **MemtoReg**
  - Next PC: PC + 4 vs. branch target → **Branch AND Zero** (PCSrc)
  - Write register number: **Rt vs Rd** → **RegDst**

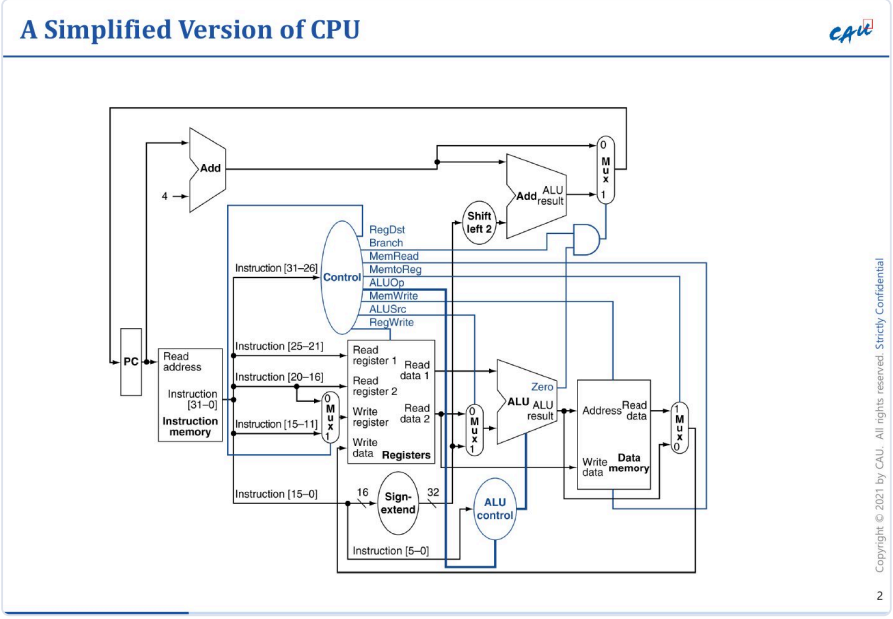
Tracing the Full Datapath (R-type · lw · sw · beq)

Slides: week6 p.3, p.26, p.34–37

§ Where the Instruction Fields Go

Where the 32-bit instruction is split in the p.3 figure:

Field	Bits	Goes to
opcode	[31–26]	**Control**
rs	[25–21]	Read register 1 (always read)
rt	[20–16]	Read register 2 **and** input 0 of the RegDst MUX
rd	[15–11]	Input 1 of the RegDst MUX
imm	[15–0]	**Sign-extend** (16 → 32)
funct	[5–0]	**ALU control**



week6 p.3 · A Simplified Version of CPU (the slide the professor said to understand 'completely, this one page')

§ R-type (add $t0, $s1, $s2)

- opcode 000000 → Control outputs the R-type signals.
- Read rs and rt → both ALU inputs are **registers** → **ALUSrc = 0**.
- Write the result to **rd (the third register)** → the RegDst MUX selects [15–11] → **RegDst = 1** (on p.3, input 1 is rd).
- The write data is the **ALU result** → **MemtoReg = 0**. **RegWrite = 1**.
- No memory access → MemRead = MemWrite = 0. Branch = 0. ALUOp = 10 (funct determines the operation).

§ lw ($t0, offset($s1))

- Read rs = base, sign-extend the constant offset → the second ALU input is the **constant** → **ALUSrc = 1**.
- The ALU **adds** to compute the address (ALUOp = 00). The ALU result is an **address**, not a value.
- Read data memory → **MemRead = 1**.
- Write the loaded value to a register → **MemtoReg = 1**, **RegWrite = 1**.
- The register written is **rt (the second one)** → **RegDst = 0**.

§ sw ($t0, offset($s1))

- Address calculation is the same as lw → **ALUSrc = 1**, ALUOp = 00.
- The value of rt (Read data 2) goes to the data memory's **Write data** → **MemWrite = 1**.
- No register is written → **RegWrite = 0**, so RegDst and MemtoReg are **X (don't care)**.

§ beq (rs, rt, offset)

- opcode → Control sets **Branch = 1**.
- Compare the two registers by **subtracting** → both ALU inputs are registers → **ALUSrc = 0**, ALUOp = 01 (subtract).
- If equal, ALU **Zero = 1** → **AND(Branch, Zero) = 1** → the PC MUX selects the **branch target (input 1)**. Otherwise 0 → **PC + 4 (input 0)**.
- Branch target: sign-extend the offset → shift left 2 → add to (PC + 4).
- No register write → RegWrite = 0, RegDst = X, MemtoReg = X. No memory access.

§ The MUX Number Trap (p.3 MemtoReg)

In the p.3 figure, three of the MUXes have **0 on top**, but the rightmost **MemtoReg MUX has 1 on top** (Read data) and **0 on the bottom** (ALU result).

- The professor explained: "0 to select the ALU result, 1 to select the data memory value" → matches the numbers in the figure. Look at the **numbers**, not top/bottom.
- The professor: "if I flip the numbers on your multiplexer … you will have to generate a total opposite signal". If the exam figure swaps the numbers, the answer changes too.

> Exam point: For a MUX question: ① find the input wire you want ② the number written on that wire = the signal value. Do not just write down a memorized table.

§ Branch Offset Calculation (board example)

Offset = (Target − (PC + 4)) / 4, **Target = (PC + 4) + Offset × 4**

Week 6-2 board: PC = 108 → PC + 4 = 112, target = 124

```
(124 - 112) / 4 = 3          → beq rs, rt, 3
3 (16-bit) → sign-extend → 0000 … 0011 (32-bit)
shift left 2 → 12
112 + 12 = 124  ✓
```

Week 6-1 board: beq at address 100, target 180 → (180 − 104) / 4 = 76 / 4 = **19**.

- If the target is above (earlier code), the offset is **negative** → sign extension fills the upper bits with 1s.

ALU Control

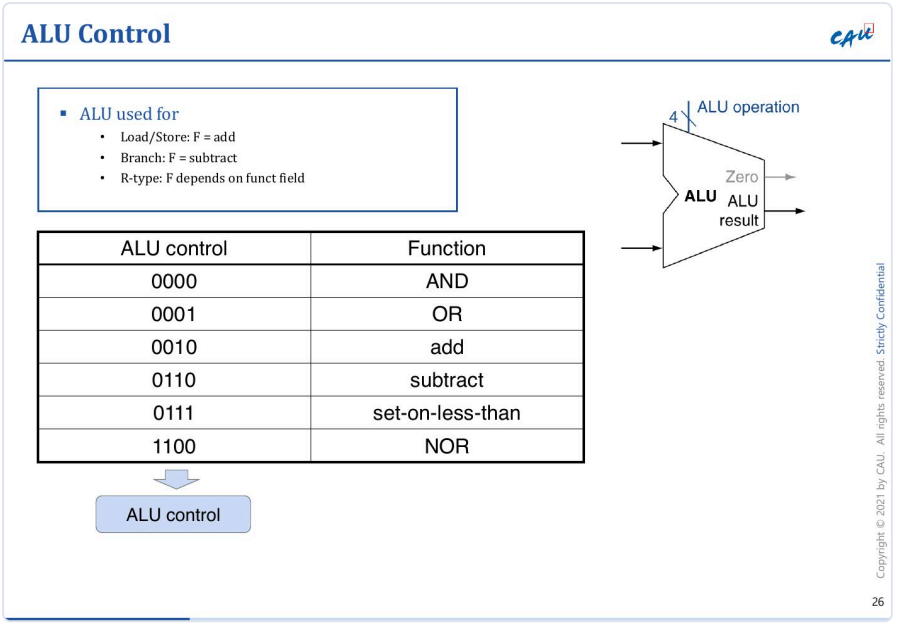
Slides: week6 p.27–30

§ Uses of the ALU and the ALU Control Codes

- Load/Store: F = **add** (address calculation)
- Branch: F = **subtract** (equality test)
- R-type: F depends on the **funct field**

ALU control	Function
0000	AND
0001	OR
0010	add
0110	subtract
0111	set-on-less-than
1100	NOR

> Exam point: The professor said "you do not need to memorize this table. It will be given on the exam. But you must be able to **read it and apply it to the circuit**." (Week 6-2 37:31; which table he pointed to is inferred from the recording.)



week6 p.27 · ALU control codes

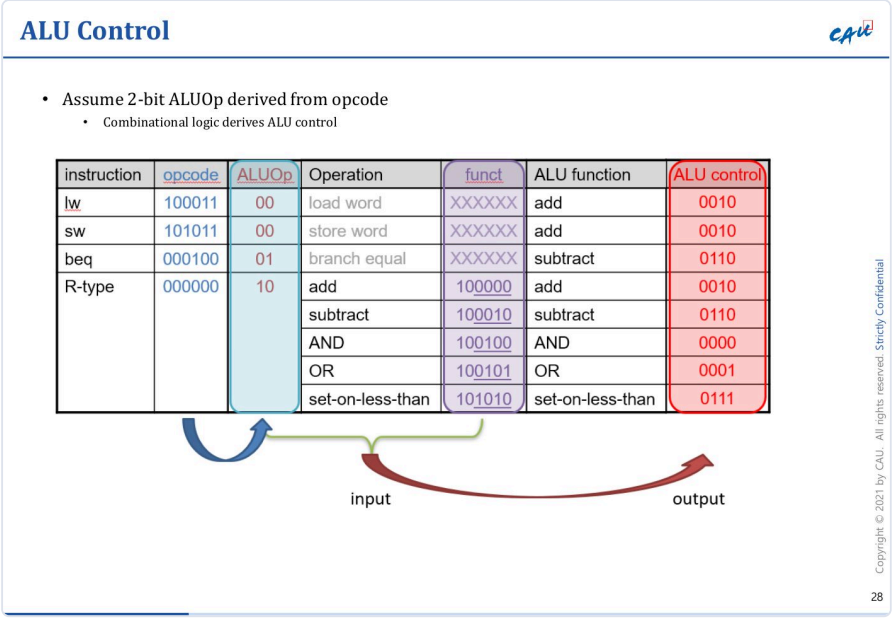
§ Generating ALU Control from ALUOp and funct

The 2-bit **ALUOp** is derived from the opcode (by the Main control), and the ALU control unit uses **ALUOp + funct (6 bits)** to produce the 4-bit ALU control (combinational logic).

opcode	ALUOp	Operation	funct	ALU function	ALU control
lw	00	load word	XXXXXX	add	0010
sw	00	store word	XXXXXX	add	0010
beq	01	branch equal	XXXXXX	subtract	0110
R-type	10	add	100000	add	0010
		subtract	100010	subtract	0110
		AND	100100	AND	0000
		OR	100101	OR	0001
		set-on-less-than	101010	set-on-less-than	0111

- lw/sw/beq are I-type, so they **have no funct** → decided by ALUOp alone (00 → add, 01 → subtract).
- When ALUOp = 10, the ALU control **waits for funct** to decide (add = 32 = 100000).

Trap: All R-type funct codes start with `10` (100000, 100010, ...), the same first two bits as the R-type ALUOp `10`. The professor pointed this out: "don't get confused" (Week 6-2 42:08).



week6 p.29 · ALUOp + funct → ALU control

§ ALU Control Logic from the K-map

Treating the ALU control table as a truth table and simplifying with a K-map gives:

```
Op3 = 0
Op2 = ALUOp0 + ALUOp1·F1
Op1 = ¬ALUOp1 + ¬F2
Op0 = ALUOp1·(F3 + F0)
```

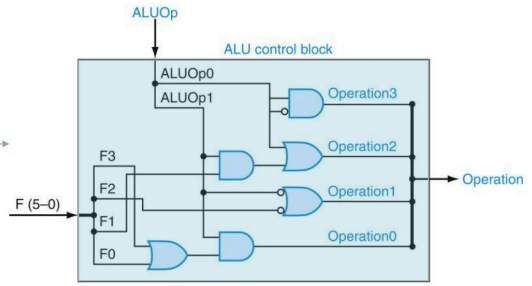
Trap: Slide p.30 is missing the NOT bars, so it reads `Op1 = ALUOp1 + F2` (and `APUOp0` in the Op3 line is also a typo). Used as written, lw (ALUOp 00) gives Op1 = 0, which does not match the table (0010). The equations above were checked against all 8 rows of the table.

ALU Control

ALUOp		Funct field						Operation
ALUOp1	ALUOp0	F5	F4	F3	F2	F1	F0	
0	0	X	X	X	X	X	X	0010
0	1	X	X	X	X	X	X	0110
1	X	X	X	0	0	0	0	0010
1	X	X	X	0	0	1	0	0110
1	X	X	X	0	1	0	0	0000
1	X	X	X	0	1	0	1	0001
1	X	X	X	1	0	1	0	0111

Karnough-Map!

Op3 = always 0
= ALUOp0 · APUOp0
Op2 = ALUOp0
+ ALUOp1 · F1
Op1 = ALUOp1 + F2
Op0 = ALUOp1 · (F3 + F0)



Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

29

week6 p.30 · K-map (watch for the missing NOT bars in the equations)

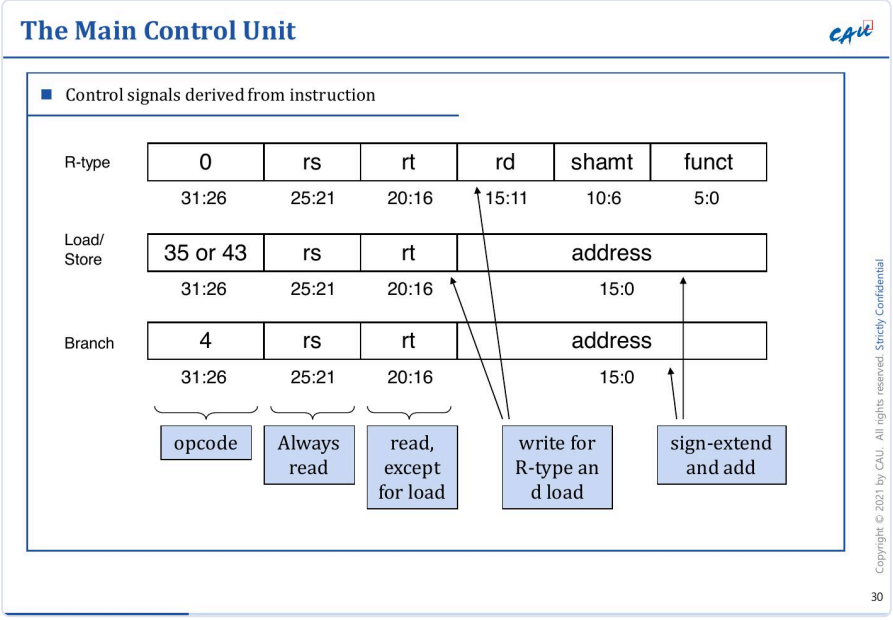
The Main Control Unit · Control Signal Table

Slides: week6 p.31–33

§ Instruction Formats and Control

	[31:26]	[25:21]	[20:16]	[15:11]	[10:6]	[5:0]
R-type	0	rs	rt	rd	shamt	funct
Load/Store	35 or 43	rs	rt	address (15:0)		
Branch	4	rs	rt	address (15:0)		

- **opcode** → input to Control
- **rs: always read**
- **rt: read, except for load** (where it is the write destination)
- **rd: write for R-type**... load writes to rt → RegDst MUX
- **address: sign-extend and add**



week6 p.31 · The Main Control Unit

§ The Control Signal Table

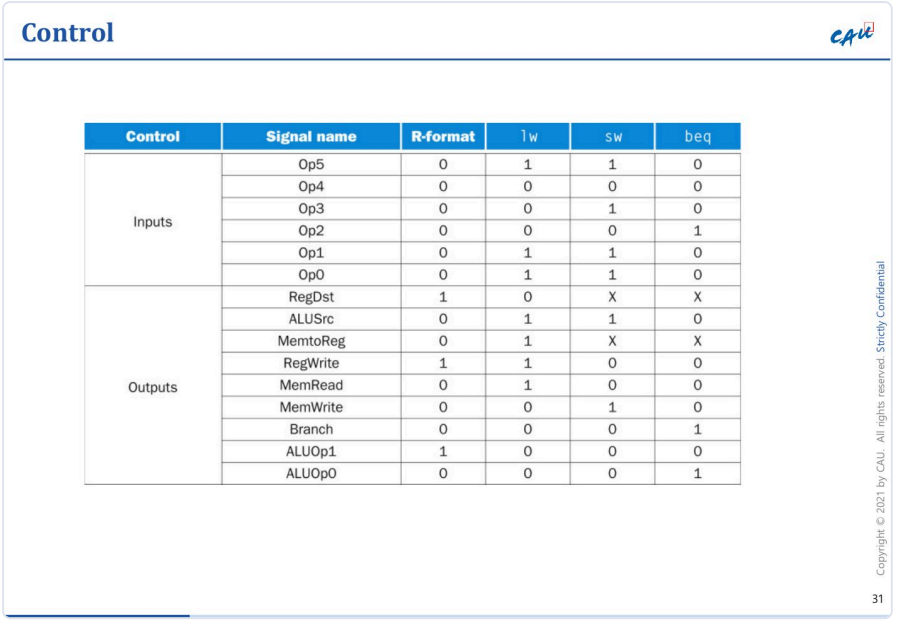
Signal	R-format	lw	sw	beq
RegDst	1	0	X	X
ALUSrc	0	1	1	0
MemtoReg	0	1	X	X
RegWrite	1	1	0	0
MemRead	0	1	0	0
MemWrite	0	0	1	0

Signal	R-format	lw	sw	beq
Branch	0	0	0	1
ALUOp1	1	0	0	0
ALUOp0	0	0	0	1

Inputs (opcode Op5–Op0): R-format 000000, lw 100011 (35), sw 101011 (43), beq 000100 (4).

Don't memorize; derive each value **from the reason**:

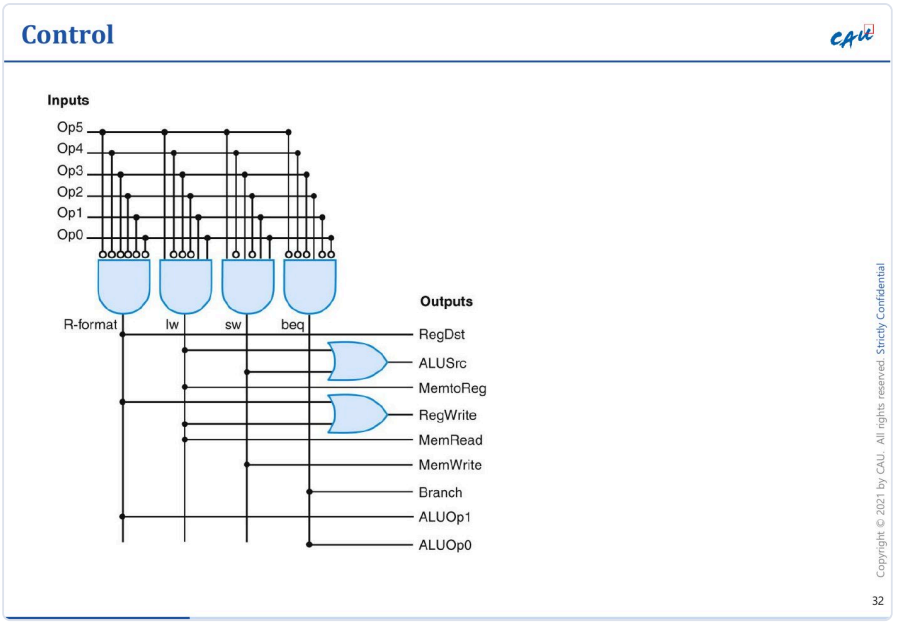
- **X appears in RegDst and MemtoReg of instructions that do not write a register (sw, beq)**. Since nothing is written, which register or which data does not matter.
- ALUSrc = 1 ⇔ lw, sw, which use a constant (offset).
- RegWrite = 1 ⇔ R-type, lw, which write their result to a register.



week6 p.32 · Control signal table

§ The Control Unit Circuit (p.33)

Four AND gates each decode the 6 opcode bits (Op5–Op0) into **R-format / lw / sw / beq**, and OR gates form the output signals. E.g., ALUSrc = lw OR sw, RegWrite = R-format OR lw. RegDst = R-format, MemtoReg = lw, MemRead = lw, MemWrite = sw, Branch = beq, ALUOp1 = R-format, ALUOp0 = beq.



week6 p.33 · Control unit circuit

§ The lw/sw Offset Is a Constant (repeated by the professor)

- lw/sw are **I-type**, so the offset must be a **constant**.
- On the exam, a statement that gives the offset as a register value → **False**.
- To use an offset held in a register: first compute base + offset into a register, then use **constant 0** as the offset, as in `lw $t0, 0($t1)`.

> Exam point: The same warning in Weeks 3, 4 and 6: "you can never just provide your offset which is contained inside the register directly into an I-type instruction".